PRAXISHANDBUCH

Linux Security

Band 2: Hardening

Linux-Server professionell härten – von nftables bis Container-Sicherheit



SECHZEHN KAPITEL. EIN DURCHGÄNGIGES SICHERHEITSKONZEPT.

Mit 32 Laborübungen, Kontrollfragen und Lernzielkontrollen · Ausgabe Oktober 2026

Linux Security
Band 2: Hardening

Linux-Server professionell härten – von nftables bis Container-Sicherheit

Von AppArmor und Fail2ban über Auditierung und Container-Sicherheit bis zum Unternehmensserver.
Mit 32 Laborübungen auf Debian 13, Kontrollfragen und Lernzielkontrollen je Kapitel.

Inhaltsverzeichnis

Vorwort 8

Die Laborumgebung dieses Bandes 8

Aufbau dieses Bandes 8

Kapitel 1: Linux Security Hardening 9

1. Einführung 9
2. Technische Grundlagen 9
3. Fachbegriffe 9
4. Praxis und Konfiguration 9
5. Sicherheitsbetrachtung 10
6. Praktische Laborübungen 10
7. Typische Fehler 11
8. Zusammenfassung 11
9. Kontrollfragen 11
10. Lernzielkontrolle 11

Kapitel 2: Sichere Benutzer- und Rechteverwaltung 12

1. Einführung 12
2. Technische Grundlagen 12
3. Fachbegriffe 12
4. Praxis und Konfiguration 12
5. Sicherheitsbetrachtung 13
6. Praktische Laborübungen 13
7. Typische Fehler 14
8. Zusammenfassung 14
9. Kontrollfragen 14
10. Lernzielkontrolle 14

Kapitel 3: SSH-Hardening 15

1. Einführung 15
2. Technische Grundlagen 15
3. Fachbegriffe 15
4. Praxis und Konfiguration 15
5. Sicherheitsbetrachtung 16
6. Praktische Laborübungen 16

7. Typische Fehler . . . 16
8. Zusammenfassung . . . 17
9. Kontrollfragen . . . 17
10. Lernzielkontrolle . . . 17

Kapitel 4: Firewall-Konfiguration mit nftables . . . 18

1. Einführung . . . 18
2. Technische Grundlagen . . . 18
3. Fachbegriffe . . . 18
4. Praxis und Konfiguration . . . 19
5. Sicherheitsbetrachtung . . . 19
6. Praktische Laborübungen . . . 19
7. Typische Fehler . . . 20
8. Zusammenfassung . . . 20
9. Kontrollfragen . . . 20
10. Lernzielkontrolle . . . 21

Kapitel 5: UFW in der Praxis . . . 22

1. Einführung . . . 22
2. Technische Grundlagen . . . 22
3. Fachbegriffe . . . 22
4. Praxis und Konfiguration . . . 22
5. Sicherheitsbetrachtung . . . 23
6. Praktische Laborübungen . . . 23
7. Typische Fehler . . . 23
8. Zusammenfassung . . . 24
9. Kontrollfragen . . . 24
10. Lernzielkontrolle . . . 24

Kapitel 6: AppArmor und Mandatory Access Control . . . 25

1. Einführung . . . 25
2. Technische Grundlagen . . . 25
3. Fachbegriffe . . . 25
4. Praxis und Konfiguration . . . 25
5. Sicherheitsbetrachtung . . . 26
6. Praktische Laborübungen . . . 26
7. Typische Fehler . . . 27
8. Zusammenfassung . . . 27

9. Kontrollfragen . . . 27
10. Lernzielkontrolle . . . 27

Kapitel 7: Fail2ban und Angriffsschutz . . . 28

1. Einführung . . . 28
2. Technische Grundlagen . . . 28
3. Fachbegriffe . . . 28
4. Praxis und Konfiguration . . . 28
5. Sicherheitsbetrachtung . . . 29
6. Praktische Laborübungen . . . 29
7. Typische Fehler . . . 30
8. Zusammenfassung . . . 30
9. Kontrollfragen . . . 30
10. Lernzielkontrolle . . . 31

Kapitel 8: Systemdienste und Prozesssicherheit . . . 32

1. Einführung . . . 32
2. Technische Grundlagen . . . 32
3. Fachbegriffe . . . 32
4. Praxis und Konfiguration . . . 32
5. Sicherheitsbetrachtung . . . 33
6. Praktische Laborübungen . . . 33
7. Typische Fehler . . . 34
8. Zusammenfassung . . . 34
9. Kontrollfragen . . . 34
10. Lernzielkontrolle . . . 34

Kapitel 9: Kernel-Sicherheit . . . 35

1. Einführung . . . 35
2. Technische Grundlagen . . . 35
3. Fachbegriffe . . . 35
4. Praxis und Konfiguration . . . 35
5. Sicherheitsbetrachtung . . . 36
6. Praktische Laborübungen . . . 36
7. Typische Fehler . . . 37
8. Zusammenfassung . . . 37
9. Kontrollfragen . . . 37
10. Lernzielkontrolle . . . 37

Kapitel 10: Auditierung mit auditd . . . 38

1. Einführung . . . 38
2. Technische Grundlagen . . . 38
3. Fachbegriffe . . . 38
4. Praxis und Konfiguration . . . 38
5. Sicherheitsbetrachtung . . . 39
6. Praktische Laborübungen . . . 39
7. Typische Fehler . . . 40
8. Zusammenfassung . . . 40
9. Kontrollfragen . . . 40
10. Lernzielkontrolle . . . 40

Kapitel 11: Integritätsüberwachung mit AIDE . . . 41

1. Einführung . . . 41
2. Technische Grundlagen . . . 41
3. Fachbegriffe . . . 41
4. Praxis und Konfiguration . . . 41
5. Sicherheitsbetrachtung . . . 42
6. Praktische Laborübungen . . . 42
7. Typische Fehler . . . 43
8. Zusammenfassung . . . 43
9. Kontrollfragen . . . 43
10. Lernzielkontrolle . . . 43

Kapitel 12: Malware-Erkennung mit ClamAV . . . 44

1. Einführung . . . 44
2. Technische Grundlagen . . . 44
3. Fachbegriffe . . . 44
4. Praxis und Konfiguration . . . 44
5. Sicherheitsbetrachtung . . . 45
6. Praktische Laborübungen . . . 45
7. Typische Fehler . . . 46
8. Zusammenfassung . . . 46
9. Kontrollfragen . . . 46
10. Lernzielkontrolle . . . 46

Kapitel 13: Backup- und Wiederherstellungskonzepte . . . 47

1. Einführung . . . 47

2. Technische Grundlagen . . . 47
3. Fachbegriffe . . . 47
4. Praxis und Konfiguration . . . 47
5. Sicherheitsbetrachtung . . . 48
6. Praktische Laborübungen . . . 48
7. Typische Fehler . . . 49
8. Zusammenfassung . . . 49
9. Kontrollfragen . . . 49
10. Lernzielkontrolle . . . 49

Kapitel 14: Docker- und Container-Sicherheit . . . 51

1. Einführung . . . 51
2. Technische Grundlagen . . . 51
3. Fachbegriffe . . . 51
4. Praxis und Konfiguration . . . 51
5. Sicherheitsbetrachtung . . . 52
6. Praktische Laborübungen . . . 52
7. Typische Fehler . . . 53
8. Zusammenfassung . . . 53
9. Kontrollfragen . . . 53
10. Lernzielkontrolle . . . 53

Kapitel 15: Webserver-Sicherheit . . . 54

1. Einführung . . . 54
2. Technische Grundlagen . . . 54
3. Fachbegriffe . . . 54
4. Praxis und Konfiguration . . . 54
5. Sicherheitsbetrachtung . . . 55
6. Praktische Laborübungen . . . 55
7. Typische Fehler . . . 56
8. Zusammenfassung . . . 56
9. Kontrollfragen . . . 56
10. Lernzielkontrolle . . . 57

Kapitel 16: Sicherheitskonzepte für Unternehmensserver . . . 58

1. Einführung . . . 58
2. Technische Grundlagen . . . 58
3. Fachbegriffe . . . 58

4. Praxis und Konfiguration 58
5. Sicherheitsbetrachtung 59
6. Praktische Laborübungen 59
7. Typische Fehler 60
8. Zusammenfassung 60
9. Kontrollfragen 60
10. Lernzielkontrolle 60
Abschlussprojekt Band 2: Professionelles Server-Hardening 61

Verzeichnis der Befehlsbeispiele 62

Index 64

Vorwort

Band 1 dieser Reihe hat den Grundserver aus dem Abschlussprojekt aufgebaut: Benutzerverwaltung, schlüsselbasiertes SSH, eine einfache Standard-Deny-Firewall, automatisierte Sicherheitsupdates, Protokollierung und die Grundlagen der Verschlüsselung. Band 2 härtet genau diesen Server weiter, mit Werkzeugen und Verfahren, die über die Grundlagen hinausgehen: Mandatory Access Control mit AppArmor, automatisierte Angriffsabwehr mit Fail2ban, Auditierung, Integritätsüberwachung, Malware-Erkennung, Backup-Konzepte und die besonderen Anforderungen von Containern und Webservern.

Dieses Buch setzt Band 1 als bekannt voraus. Begriffe wie UID/GID, PAM, die grundlegende nftables-Syntax oder der Unterschied zwischen Hashing und Verschlüsselung werden hier nicht erneut erklärt.

Die Laborumgebung dieses Bandes

Band 2 nutzt dieselbe Laborumgebung wie Band 1: `security-client` (Kali Linux, 192.168.56.10) und `server1` (Debian 13, 192.168.56.20). Server 2 und der Windows-11-Client aus dem Master-Lab kommen erst in Band 3 hinzu, sobald Netzwerkanalyse und plattformübergreifende Szenarien eine Rolle spielen.

> **WARNUNG – Sicherheitsrisiko**
>
> `server1` sollte zu Beginn von Band 2 im Zustand des Abschlussprojekts aus Band 1 vorliegen (Benutzerkonzept, SSH-Härtung, Firewall, automatisierte Updates). Lege an dieser Stelle einen neuen Snapshot an, bevor du mit den Übungen dieses Bandes beginnst – mehrere Kapitel verändern dieselbe Konfiguration nacheinander.

Aufbau dieses Bandes

Band 2 behandelt sechzehn Kapitel, gegliedert in vier thematische Abschnitte: Systemhärtung und Zugriffskontrolle (Kapitel 1-6), Angriffsschutz und Überwachung (Kapitel 7-11), Betrieb und Wiederherstellung (Kapitel 12-13) sowie Anwendungs- und Unternehmenssicherheit (Kapitel 14-16). Jedes Kapitel folgt derselben, aus Band 1 bekannten zehnteiligen Struktur.

Kapitel 1: Linux Security Hardening

Lernziele

- Du erklärst, was Hardening systematisch von einer Einzelmaßnahme unterscheidet.
- Du nutzt einen anerkannten Hardening-Standard (CIS Benchmark) als Orientierung.
- Du legst eine eigene, nachvollziehbare Hardening-Dokumentation an.

1. Einführung

Band 1 hat einzelne Sicherheitsmaßnahmen eingeführt: ein Konto hier, eine Firewall-Regel dort. **Hardening** bündelt diese Einzelmaßnahmen zu einem systematischen, nachvollziehbaren Prozess, der sich wiederholen und überprüfen lässt. Dieses Kapitel legt den gemeinsamen Rahmen für alle folgenden fünfzehn Kapitel.

2. Technische Grundlagen

Hardening-Standards fassen bewährte Einzelmaßnahmen zu geprüften, oft nach Risiko gestaffelten Empfehlungskatalogen zusammen. Der bekannteste ist das **CIS Benchmark** für Debian Linux, das Hunderte Einzelprüfungen in Abschnitte wie Dateisystem, Dienste, Netzwerk und Protokollierung gliedert, jeweils mit einer begründeten Empfehlung und einer Einstufung nach Wirkungsgrad (Level 1 für breit anwendbare, Level 2 für umgebungsspezifischere Maßnahmen).

Beispiel 1.1: Der Hardening-Kreislauf (Konzept)

```
Baseline erfassen -> gegen Standard prüfen -> Abweichungen beheben
     -> dokumentieren -> erneut prüfen (wiederkehrend, nicht einmalig)
```

Hardening ist damit kein einmaliger Zustand, sondern ein wiederkehrender Prozess: Jede neue Softwareversion, jede neue Konfiguration und jede neue Bedrohung kann eine bereits geprüfte Baseline wieder verändern.

3. Fachbegriffe

Eine **Baseline** ist der dokumentierte Ausgangszustand eines Systems zu einem bestimmten Zeitpunkt, gegen den sich spätere Änderungen vergleichen lassen. Ein **Benchmark** ist ein strukturierter Katalog von Prüfpunkten mit jeweils einer Sollvorgabe. **Defense in Depth** bezeichnet das Prinzip, Sicherheit nicht auf eine einzelne Maßnahme zu stützen, sondern mehrere, unabhängig wirkende Schichten hintereinanderzuschalten, sodass der Ausfall einer Schicht nicht automatisch das gesamte System gefährdet.

4. Praxis und Konfiguration

Beispiel 1.2: Eine einfache Baseline-Dokumentation beginnen

```
$ dpkg -l > baseline_pakete.txt
$ ss -tulpen > baseline_ports.txt
$ systemctl list-unit-files --state=enabled > baseline_dienste.txt
```

Beispiel 1.3: Einen Prüfpunkt manuell gegen eine Empfehlung abgleichen (Konzept)

```
CIS-Empfehlung: "Stelle sicher, dass unbenutzte Dateisystem-Kernelmodule deaktiviert sind"
Prüfbefehl:     $ lsmod | grep cramfs
Sollvorgabe:    kein Treffer, wenn das Modul nicht benötigt wird
```

Diese Art des manuellen Abgleichs ist mühsam, aber lehrreich: Sie zeigt, wie ein Hardening-Standard aufgebaut ist, bevor Band 3 mit Lynis ein Werkzeug einführt, das denselben Abgleich automatisiert.

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Nicht jede Benchmark-Empfehlung passt auf jede Umgebung. Level-2-Empfehlungen können die Funktionalität eines Systems einschränken, wenn sie unreflektiert übernommen werden. Jede Abweichung von einer Empfehlung sollte bewusst getroffen und dokumentiert werden, nicht stillschweigend übersprungen.

Ein häufiger Fehler ist, Hardening als abgeschlossenes Projekt statt als Prozess zu behandeln: eine einmalige Konfiguration, die danach nie wieder geprüft wird, veraltet ebenso wie ein nicht aktualisiertes Paket.

6. Praktische Laborübungen

Übung A: Eine eigene Baseline erstellen

Aufgabenstellung: Erstelle eine dokumentierte Baseline von `server1` im aktuellen Zustand. **Ausgangssituation:** `server1` im Zustand nach dem Band-1-Abschlussprojekt. **Voraussetzungen:** SSH-Zugriff. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 1.2. **Erwartetes Ergebnis:** Drei Textdateien mit Paketliste, offenen Ports und aktivierten Diensten. **Überprüfung:** Jede Datei enthält plausible, nicht leere Inhalte. **Fehleranalyse:** Leere Datei? Prüfe, ob der Befehl tatsächlich mit ausreichenden Rechten lief. **Musterlösung:** Die drei Dateien dienen als Referenzpunkt für alle folgenden Kapitel dieses Bandes.

Übung B: Eine CIS-Empfehlung manuell prüfen

Aufgabenstellung: Wähle zwei CIS-Debian-Benchmark-Empfehlungen aus und prüfe `server1` manuell dagegen. **Ausgangssituation:** wie Übung A. **Voraussetzungen:** Internetzugang von `security-client`, um den öffentlich verfügbaren Benchmark einzusehen.
Schritt-für-Schritt-Anleitung: Benchmark-Dokument sichten, zwei Prüfpunkte auswählen, jeweils den zugehörigen Prüfbefehl auf `server1` ausführen (vergleiche Beispiel 1.3). **Erwartetes Ergebnis:** Für beide Prüfpunkte eine klare Aussage: erfüllt, nicht erfüllt, oder bewusst abgewichen mit Begründung. **Überprüfung:** Die Begründung für eine Abweichung ist nachvollziehbar und nicht nur „war zu kompliziert“. **Fehleranalyse:** Prüfbefehl liefert unklares Ergebnis? Benchmark-Dokumente enthalten meist zusätzliche Erläuterungen zur Interpretation. **Musterlösung:** Eine vollständige Lösung dokumentiert beide Prüfpunkte mit Befehl, Ergebnis und Bewertung.

7. Typische Fehler

Fehler	Ursache	Behebung
Hardening als einmaliges Projekt behandelt	fehlendes Prozessverständnis	wiederkehrende Prüfungen einplanen, nicht nur einmalig durchführen
Jede Benchmark-Empfehlung unreflektiert übernommen	Bequemlichkeit statt Abwägung	Level und Kontext jeder Empfehlung prüfen, bewusst abweichen wo nötig
Keine Baseline vor Beginn der Härtung	fehlende Vergleichsgrundlage	immer zuerst dokumentieren, dann verändern

8. Zusammenfassung

Zusammenfassung

- Hardening ist ein wiederkehrender Prozess, kein einmaliger Zustand.
- CIS Benchmarks bündeln bewährte Einzelmaßnahmen in geprüften, gestaffelten Katalogen.
- Eine dokumentierte Baseline ist die Voraussetzung, um spätere Änderungen zu bewerten.
- Defense in Depth verteilt Sicherheit auf mehrere, unabhängige Schichten.

9. Kontrollfragen

1. Was ist eine Baseline? (leicht)
2. Wofür steht CIS im Zusammenhang mit Hardening-Standards? (leicht)
3. Warum ist Hardening ein Prozess und kein Zustand? (mittel)
4. Was unterscheidet Level-1- von Level-2-Empfehlungen in einem CIS Benchmark? (mittel)
5. Ein Team möchte „den CIS Benchmark vollständig umsetzen“, ohne eine einzige Abweichung zu dokumentieren. Welches Problem siehst du darin? (schwer)

Lösungshinweise: 1. Der dokumentierte Ausgangszustand eines Systems als Vergleichsgrundlage. 2. Center for Internet Security, Herausgeber verbreiteter Hardening-Benchmarks. 3. Weil sich Software, Konfiguration und Bedrohungslage laufend ändern. 4. Level 1 ist breit anwendbar mit geringem Funktionsrisiko, Level 2 greift tiefer und kann die Funktionalität einschränken. 5. Eine vollständige, unreflektierte Umsetzung ohne Abwägung kann Funktionen beeinträchtigen, die im konkreten Einsatzkontext tatsächlich gebraucht werden; bewusste, begründete Abweichungen sind oft sinnvoller als blinder Gehorsam gegenüber dem Katalog.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] eine eigene Systembaseline dokumentiert hast,
- [ ] mindestens eine CIS-Empfehlung manuell geprüft hast,
- [ ] erklären kannst, warum Hardening ein Prozess ist,
- [ ] eine bewusste Abweichung von einer Empfehlung begründen kannst.

Kapitel 2: Sichere Benutzer- und Rechteverwaltung

Lernziele

- Du findest und entschärfst unnötige SUID-Binaries auf einem produktiven System.
- Du härtest `sudoers`-Regeln nach dem Prinzip der geringsten Rechte.
- Du richtest Passwort-Ablaufregeln für Bestandskonten ein.

1. Einführung

Band 1 hat Benutzer, Gruppen und `sudo` eingeführt. Dieses Kapitel geht einen Schritt weiter: Es prüft einen bereits laufenden Server auf Rechtevergaben, die sich über die Zeit angesammelt haben, und härtet sie gezielt.

2. Technische Grundlagen

Ein System sammelt über seine Lebensdauer Rechtevergaben an, die zum Einrichtungszeitpunkt sinnvoll waren, später aber nicht mehr gebraucht werden: ein Testkonto, eine zu breite `sudoers`-Regel, ein unnötiges SUID-Programm aus einer inzwischen entfernten Anwendung. Systematisches Prüfen deckt diese Altlasten auf.

Beispiel 2.1: Alle SUID-Programme eines Systems auflisten

```
$ find / -perm -4000 -type f 2>/dev/null
```

Diese Liste enthält auf einem Standard-Debian-System vorwiegend bekannte, notwendige Programme (`passwd`, `sudo`, `mount`). Jeder zusätzliche, aus einer später installierten Anwendung stammende Eintrag verdient eine gezielte Prüfung, ob das SUID-Bit tatsächlich noch gebraucht wird.

3. Fachbegriffe

Passwort-Aging bezeichnet die zeitliche Begrenzung der Gültigkeit eines Passworts, konfiguriert über `chage`. Eine **Rechte-Revision** (Access Review) ist die wiederkehrende, systematische Prüfung bestehender Rechtevergaben auf fortbestehende Notwendigkeit. Das in Band 1 eingeführte **Prinzip der geringsten Rechte** wird hier erstmals auf ein bereits gewachsenes, nicht mehr frisches System angewendet statt auf eine Neueinrichtung.

4. Praxis und Konfiguration

Beispiel 2.2: Ein unnötiges SUID-Bit entfernen

```
$ sudo chmod u-s /pfad/zum/programm
```

Beispiel 2.3: Passwort-Ablauf für ein Konto konfigurieren

```
$ sudo chage -M 90 -W 7 testuser
$ chage -l testuser
```

Option	Bedeutung
`-M 90`	Passwort läuft nach 90 Tagen ab
`-W 7`	Warnung 7 Tage vor Ablauf
`-l`	aktuelle Einstellungen anzeigen

Beispiel 2.4: sudoers-Regel verschärfen (mit visudo)

```
# Vorher: testuser ALL=(ALL) ALL
# Nachher:
testuser ALL=(root) NOPASSWD: /usr/bin/systemctl restart nginx
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Entferne niemals das SUID-Bit von Kernprogrammen wie `passwd`, `mount` oder `sudo` selbst – diese benötigen es für ihre reguläre Funktion. Prüfe vor jeder Änderung gezielt, ob ein Programm zur Grundausstattung gehört oder aus einer später installierten Anwendung stammt.

Eine zu lange Passwort-Gültigkeit bei gleichzeitig fehlender Mehr-Faktor-Authentifizierung (Band 1, Kapitel 4) verlängert das Zeitfenster, in dem ein einmal kompromittiertes Passwort nutzbar bleibt. Zu häufiger erzwungener Wechsel führt dagegen erfahrungsgemäß zu schwächeren, leicht vorhersagbaren Passwortmustern – eine Abwägung, die in der Fachwelt kontrovers diskutiert wird.

6. Praktische Laborübungen

Übung A: SUID-Bestand prüfen und dokumentieren

Aufgabenstellung: Erstelle eine vollständige SUID-Liste und ordne jeden Eintrag als „erwartet“ oder „prüfenswert“ ein. **Ausgangssituation:** `server1` nach Band 1. **Voraussetzungen:** SSH-Zugriff. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 2.1. **Erwartetes Ergebnis:** Eine Liste mit Einordnung für jeden Fund. **Überprüfung:** Kein Eintrag bleibt ohne Einordnung. **Fehleranalyse:** Unklarer Eintrag? Paketzugehörigkeit mit `dpkg -S /pfad/zum/programm` klären. **Musterlösung:** Auf einem Standardsystem sind praktisch alle Einträge erwartet; ein Fund ohne erkennbaren Grund wird in Übung B behandelt.

Übung B: sudoers-Regel verschärfen und testen

Aufgabenstellung: Ersetze eine zu breite `sudoers`-Regel durch eine eng begrenzte, nach Beispiel 2.4. **Ausgangssituation:** `testuser` besitzt noch die breite Regel aus Band 1. **Voraussetzungen:** sudo-Rechte, zweite offene Sitzung. **Schritt-für-Schritt-Anleitung:** `visudo` öffnen, Regel wie in Beispiel 2.4 ersetzen. **Erwartetes Ergebnis:** `testuser` kann weiterhin `systemctl restart nginx` ohne Passwort ausführen, aber keine anderen Befehle mehr als root. **Überprüfung:** `sudo -l` als `testuser` zeigt ausschließlich die eine, eng begrenzte Regel. **Fehleranalyse:** Befehl funktioniert nicht? Prüfe den exakten Pfad des Programms mit `which systemctl`. **Musterlösung:** `sudo -l` bestätigt die eingeschränkte Regel, ein Versuch mit einem anderen Befehl wird abgelehnt.

7. Typische Fehler

Fehler	Ursache	Behebung
SUID-Bit von Kernprogramm entfernt	fehlende Unterscheidung Grundausstattung/Zusatzsoftware	Paketzugehörigkeit vor jeder Änderung prüfen
sudoers-Regel „aus Bequemlichkeit“ breit gelassen	kurzfristiger Aufwand gescheut	Regel beim ersten Einrichten bereits eng formulieren
Passwort-Aging ohne Vorwarnung aktiviert	`-W` vergessen	Warnfrist immer mit konfigurieren

8. Zusammenfassung

Zusammenfassung

- Gewachsene Systeme sammeln Rechtevergaben an, die regelmäßig überprüft werden sollten.
- `find / -perm -4000` macht den SUID-Bestand eines Systems vollständig sichtbar.
- sudoers-Regeln sollten so eng wie möglich formuliert werden.
- Passwort-Aging ist ein zweischneidiges Werkzeug und sollte mit Bedacht konfiguriert werden.

9. Kontrollfragen

1. Welcher Befehl listet alle SUID-Programme eines Systems auf? (leicht)
2. Wozu dient `chage`? (leicht)
3. Warum sollte das SUID-Bit von `passwd` niemals entfernt werden? (mittel)
4. Was ist eine Rechte-Revision? (mittel)
5. Ein Sicherheitsteam erzwingt einen monatlichen Passwortwechsel für alle Konten. Welches Gegenargument aus der Praxis könntest du anbringen? (schwer)

Lösungshinweise: 1. `find / -perm -4000 -type f`. 2. Es konfiguriert und zeigt die Passwort-Ablaufregeln eines Kontos. 3. Weil `passwd` genau dieses Recht benötigt, um Passwort-Hashes im Namen des Benutzers zu ändern. 4. Die wiederkehrende, systematische Prüfung bestehender Rechtevergaben auf fortbestehende Notwendigkeit. 5. Zu häufiger erzwungener Wechsel führt in der Praxis oft zu vorhersagbaren, leicht zu erratenden Mustern (z. B. fortlaufende Nummerierung), was dem eigentlichen Sicherheitsziel entgegenwirken kann.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] den SUID-Bestand eines Systems vollständig dokumentiert hast,
- [ ] eine zu breite sudoers-Regel gezielt eingeschränkt hast,
- [ ] Passwort-Ablaufregeln für ein Konto konfiguriert hast,
- [ ] eine begründete Haltung zu erzwungenem Passwortwechsel entwickeln kannst.

Kapitel 3: SSH-Hardening

Lernziele

- Du beschränkst SSH-Zugriff auf bestimmte Benutzer und Netzwerke.
- Du konfigurierst moderne, sichere Verschlüsselungsalgorithmen für SSH.
- Du richtest ein Zeitlimit für inaktive Verbindungen ein.

1. Einführung

Band 1 hat Schlüsselauthentifizierung und `PermitRootLogin no` eingeführt. Dieses Kapitel vertieft die SSH-Härtung um Zugriffsbeschränkung, Algorithmenauswahl und Verbindungsmanagement.

2. Technische Grundlagen

SSH erlaubt, den Zugriff nicht nur über Authentifizierungsmethoden, sondern auch über Benutzer- und Netzwerkzugehörigkeit einzuschränken. Zusätzlich lässt sich festlegen, welche kryptografischen Algorithmen ein Client überhaupt anbieten darf – ältere, als schwach geltende Algorithmen sollten explizit ausgeschlossen werden, statt sich auf veraltete Standardlisten zu verlassen.

Beispiel 3.1: Zugriff auf bestimmte Benutzer und Netzwerke beschränken

```
AllowUsers eugen@192.168.56.0/24
```

3. Fachbegriffe

Jump Host (auch Bastion Host) ist ein eigens dafür vorgesehener Server, über den sämtlicher SSH-Zugriff auf weitere, tiefer liegende Server geleitet wird, statt jeden Server einzeln direkt erreichbar zu machen. **ClientAliveInterval** und **ClientAliveCountMax** steuern gemeinsam, nach welcher Inaktivität eine SSH-Verbindung serverseitig beendet wird.

4. Praxis und Konfiguration

Beispiel 3.2: Moderne Algorithmen erzwingen (Auszug aus sshd_config)

```
KexAlgorithms curve25519-sha256
Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com
MACs hmac-sha2-512-etm@openssh.com
```

Beispiel 3.3: Inaktive Verbindungen automatisch trennen

```
ClientAliveInterval 300
ClientAliveCountMax 2
```

Diese Kombination prüft alle 300 Sekunden, ob der Client noch reagiert, und trennt die Verbindung nach zwei ausbleibenden Antworten – nach spätestens zehn Minuten Inaktivität.

Beispiel 3.4: Konfiguration testen und übernehmen

```
$ sudo sshd -t
$ sudo systemctl reload sshd
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Prüfe die von dir gewählten Algorithmen gegen die aktuelle OpenSSH-Dokumentation, bevor du veraltete Standardlisten blind übernimmst – als unsicher geltende Algorithmen ändern sich über die Zeit, eine heute empfohlene Liste kann in einigen Jahren veraltet sein.

`AllowUsers` mit einer Netzwerkeinschränkung reduziert die Angriffsfläche erheblich, setzt aber voraus, dass berechtigte Nutzer tatsächlich nur aus dem erlaubten Netzwerk zugreifen – ein Außendienstmitarbeiter mit wechselnder IP-Adresse würde ausgesperrt, sofern kein VPN oder Jump Host zwischengeschaltet ist.

6. Praktische Laborübungen

Übung A: Zugriffsbeschränkung einrichten

Aufgabenstellung: Beschränke SSH-Zugriff auf `server1` auf den Benutzer `eugen` aus dem Labornetz. **Ausgangssituation:** `server1` nach Band 1. **Voraussetzungen:** zweite offene Sitzung. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 3.1, 3.4. **Erwartetes Ergebnis:** Anmeldung als `eugen` aus dem Labornetz funktioniert, ein anderer Benutzername wird abgelehnt. **Überprüfung:** `ssh anderer@192.168.56.20` liefert „Permission denied“. **Fehleranalyse:** Niemand kommt mehr rein? Über die zweite Sitzung oder die Konsole zurücksetzen. **Musterlösung:** Nur `eugen` aus dem definierten Netz kann sich anmelden.

Übung B: Timeout für inaktive Verbindungen testen

Aufgabenstellung: Richte ein kurzes Testtimeout ein und beobachte die automatische Trennung. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** `ClientAliveInterval 10` und `ClientAliveCountMax 1` testweise setzen, verbinden, Verbindung inaktiv lassen. **Erwartetes Ergebnis:** Die Verbindung trennt sich nach etwa zehn Sekunden Inaktivität. **Überprüfung:** Die Sitzung endet mit einer Timeout-Meldung. **Fehleranalyse:** Keine Trennung? Prüfe, ob `reload` tatsächlich erfolgreich war. **Musterlösung:** Nach dem Test die produktiven Werte aus Beispiel 3.3 wiederherstellen.

7. Typische Fehler

Fehler	Ursache	Behebung
Berechtigter Nutzer ausgesperrt	`AllowUsers` ohne Berücksichtigung wechselnder IPs	Jump Host oder VPN statt starrer IP-Einschränkung erwägen
Veraltete Algorithmen weiterhin erlaubt	Standardkonfiguration nie überprüft	Algorithmenliste regelmäßig gegen aktuelle Empfehlungen prüfen
Timeout zu aggressiv für normale Arbeit	Werte ungetestet übernommen	Werte im eigenen Labor austesten, bevor sie produktiv gesetzt werden

8. Zusammenfassung

> **Zusammenfassung**
>
> - `AllowUsers` beschränkt SSH-Zugriff auf bestimmte Konten und Netzwerke.
> - Moderne Algorithmen sollten explizit erzwungen, veraltete ausgeschlossen werden.
> - `ClientAliveInterval`/`ClientAliveCountMax` beenden inaktive Verbindungen automatisch.
> - Jede Zugriffsbeschränkung sollte reale Nutzungsmuster berücksichtigen, bevor sie produktiv wird.

9. Kontrollfragen

1. Welcher Parameter beschränkt SSH-Zugriff auf bestimmte Benutzer? (leicht)
2. Wozu dient ein Jump Host? (leicht)
3. Was bewirken `ClientAliveInterval` und `ClientAliveCountMax` zusammen? (mittel)
4. Warum sollten Algorithmenlisten regelmäßig überprüft werden? (mittel)
5. Ein Außendienstmitarbeiter mit wechselnder IP-Adresse soll SSH-Zugriff behalten, ohne die Netzwerkeinschränkung für alle anderen zu lockern. Welche Lösung schlägst du vor? (schwer)

Lösungshinweise: 1. `AllowUsers`. 2. Er bündelt SSH-Zugriff auf weitere Server über einen einzigen, kontrollierten Einstiegspunkt. 3. Sie trennen eine Verbindung automatisch nach einer definierten Zeit der Inaktivität. 4. Weil als sicher geltende Algorithmen sich über die Zeit ändern. 5. Einen Jump Host oder VPN-Zugang einrichten, über den der Mitarbeiter mit einer festen, bekannten Adresse zugreift, statt die generelle Netzwerkeinschränkung aufzuweichen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] SSH-Zugriff erfolgreich auf einen Benutzer und ein Netzwerk beschränkt hast,
- [ ] moderne Algorithmen in `sshd_config` konfiguriert hast,
- [ ] ein Verbindungstimeout eingerichtet und getestet hast,
- [ ] eine Lösung für mobile, berechtigte Nutzer bei strikter Netzwerkeinschränkung nennen kannst.

Kapitel 4: Firewall-Konfiguration mit nftables

Lernziele

- Du strukturierst ein nftables-Regelwerk mit mehreren Chains und Sets.
- Du protokollierst gezielt verworfene Pakete.
- Du begrenzt Verbindungsraten gegen einfache Flood-Versuche.

1. Einführung

Band 1 hat eine einfache Standard-Deny-Regel mit nftables eingeführt. Dieses Kapitel vertieft die Syntax für produktionsnähere Szenarien: mehrere Dienste, wiederverwendbare Adresslisten und Schutz vor einfachen Überlastungsversuchen.

2. Technische Grundlagen

Ein **Set** in nftables ist eine benannte, wiederverwendbare Liste von Werten (z. B. IP-Adressen), auf die sich mehrere Regeln gleichzeitig beziehen können, ohne die Liste in jeder Regel zu wiederholen. Die Option `limit rate` begrenzt, wie oft eine Regel innerhalb eines Zeitraums greift – eine einfache Methode, einfache Flood-Versuche abzuschwächen.

Beispiel 4.1: Ein wiederverwendbares Set definieren

```
table inet filter {
    set vertrauenswuerdig {
        type ipv4_addr
        elements = { 192.168.56.10 }
    }
    chain input {
        type filter hook input priority 0; policy drop;
        ct state established,related accept
        iif lo accept
        ip saddr @vertrauenswuerdig tcp dport 22 accept
    }
}
```

3. Fachbegriffe

Rate Limiting begrenzt die Häufigkeit, mit der eine Regel innerhalb eines Zeitfensters greift. **Logging in nftables** schreibt für jedes Paket, das eine bestimmte Regel durchläuft, einen Protokolleintrag, bevor über Annahme oder Verwurf entschieden wird – wichtig für die spätere Auswertung (Kapitel 10).

4. Praxis und Konfiguration

Beispiel 4.2: Verworfene Pakete protokollieren

```
chain input {
    type filter hook input priority 0; policy drop;
    ct state established,related accept
    iif lo accept
    tcp dport 22 accept
    log prefix "nft-dropped: " drop
}
```

Beispiel 4.3: Verbindungsrate auf SSH begrenzen

```
tcp dport 22 ct state new limit rate 5/minute accept
tcp dport 22 ct state new drop
```

Beispiel 4.4: Protokollierte Verwürfe einsehen

```
$ journalctl -k | grep "nft-dropped"
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Zu aggressives Rate Limiting auf SSH kann auch berechtigte Nutzer bei mehreren schnellen, legitimen Verbindungsversuchen (z. B. automatisierte Deployment-Skripte) blockieren. Teste Grenzwerte gegen das tatsächliche, erwartete Nutzungsmuster, bevor sie produktiv gesetzt werden.

Protokollierung verworfener Pakete erzeugt bei einem aktiven Scan oder Flood-Versuch schnell sehr viele Einträge. Ohne eine nachgelagerte Auswertung (Band 3, SIEM-Themen) bleibt diese Protokollierung ungenutztes Datenvolumen – dasselbe Prinzip wie in Band 1, Kapitel 9, zur Protokollierung allgemein.

6. Praktische Laborübungen

Übung A: Ein Set für vertrauenswürdige Adressen einrichten

Aufgabenstellung: Beschränke SSH-Zugriff auf `server1` über ein Set, das nur `security-client` enthält. **Ausgangssituation:** `server1` mit Standard-Deny-Firewall aus Band 1. **Voraussetzungen:** sudo-Rechte, zweite offene Sitzung. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 4.1. **Erwartetes Ergebnis:** Nur `security-client` kann sich per SSH verbinden. **Überprüfung:** Ein simulierter Verbindungsversuch von einer anderen Adresse (z. B. über eine dritte, testweise verbundene VM) schlägt fehl. **Fehleranalyse:** Falsche Adresse im Set? Mit `ip addr show` auf `security-client` die tatsächliche Adresse prüfen. **Musterlösung:** `nft list ruleset` zeigt das Set mit der korrekten Adresse und die zugehörige, funktionierende Regel.

Übung B: Rate Limiting gegen SSH-Verbindungsversuche

Aufgabenstellung: Richte Rate Limiting nach Beispiel 4.3 ein und teste es mit mehreren schnellen Verbindungsversuchen. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** Regel einrichten, danach von `security-client` mehrfach schnell hintereinander verbinden. **Erwartetes Ergebnis:** Nach Erreichen der konfigurierten Grenze werden weitere Verbindungsversuche innerhalb desselben Zeitfensters verworfen. **Überprüfung:** `journalctl -k | grep nft-dropped` (sofern Logging ergänzt wurde) zeigt die verworfenen Versuche. **Fehleranalyse:** Keine Begrenzung spürbar? Prüfe, ob die Regel tatsächlich vor der allgemeinen Accept-Regel für Port 22 steht. **Musterlösung:** Nach dem Test die Regel wieder entfernen oder auf einen produktionsnäheren Wert anheben.

7. Typische Fehler

Fehler	Ursache	Behebung
Rate Limiting sperrt legitime Nutzung	Grenzwert ohne Testlauf gewählt	gegen reales Nutzungsmuster testen
Set-Adresse falsch eingetragen	Tippfehler oder veraltete IP	Adresse vor dem Eintragen verifizieren
Logging ohne Auswertung aktiviert	kein nachgelagerter Prozess eingeplant	Protokolle regelmäßig oder automatisiert prüfen (Band 3)

8. Zusammenfassung

Zusammenfassung

- Sets bündeln wiederverwendbare Adresslisten für mehrere Regeln.
- `limit rate` schwächt einfache Flood-Versuche gegen einen bestimmten Dienst ab.
- Protokollierung verworfener Pakete liefert nur dann Nutzen, wenn sie ausgewertet wird.
- Jede Beschränkung sollte gegen reale Nutzungsmuster getestet werden.

9. Kontrollfragen

1. Was ist ein Set in nftables? (leicht)
2. Wofür dient `limit rate`? (leicht)
3. Warum kann aggressives Rate Limiting auch legitime Nutzer betreffen? (mittel)
4. Wie lassen sich verworfene Pakete in nftables protokollieren? (mittel)
5. Ein automatisiertes Deployment-Skript baut innerhalb kurzer Zeit mehrere SSH-Verbindungen zu `server1` auf und wird durch dein Rate Limiting blockiert. Wie löst du den Konflikt? (schwer)

Lösungshinweise: 1. Eine benannte, wiederverwendbare Liste von Werten wie IP-Adressen. 2. Es begrenzt, wie oft eine Regel innerhalb eines Zeitfensters greift. 3. Weil auch legitime, aber schnelle Verbindungsmuster dieselbe Grenze erreichen können wie ein Angriff. 4. Mit der `log`-Anweisung vor der verwerfenden Regel. 5. Entweder die Adresse des Deployment-Systems über ein Set von der Begrenzung ausnehmen, oder den Grenzwert so anpassen, dass er das erwartete, legitime Muster weiterhin zulässt.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] ein Set für vertrauenswürdige Adressen eingerichtet hast,
- [ ] Rate Limiting gegen einen Dienst konfiguriert und getestet hast,
- [ ] verworfene Pakete protokollieren und auslesen kannst,
- [ ] einen Zielkonflikt zwischen Sicherheit und legitimer Nutzung lösen kannst.

Kapitel 5: UFW in der Praxis

Lernziele

- Du richtest UFW-Regeln für mehrere Dienste gleichzeitig ein.
- Du nutzt Anwendungsprofile statt einzelner Portnummern.
- Du aktivierst UFW-Logging und liest es gezielt aus.

1. Einführung

Band 1 hat UFW als einfache Alternative zu nftables vorgestellt. Dieses Kapitel zeigt UFW im alltäglichen, mehrdienstigen Praxiseinsatz, als Gegenstück zur direkten nftables-Konfiguration aus Kapitel 4.

2. Technische Grundlagen

UFW verwaltet **Anwendungsprofile**, vordefinierte Regelsätze für bekannte Dienste (z. B. „OpenSSH“, „Apache Full“), sodass nicht jede Portnummer einzeln nachgeschlagen werden muss. Diese Profile liegen unter `/etc/ufw/applications.d/` und lassen sich auch für eigene Anwendungen ergänzen.

Beispiel 5.1: Verfügbare Anwendungsprofile anzeigen

```
$ sudo ufw app list
```

3. Fachbegriffe

Ein **Anwendungsprofil** fasst die für einen Dienst nötigen Port- und Protokollregeln unter einem sprechenden Namen zusammen. **UFW-Logging** protokolliert Firewall-Entscheidungen wahlweise in unterschiedlicher Ausführlichkeit (`low`, `medium`, `high`, `full`).

4. Praxis und Konfiguration

Beispiel 5.2: Mehrere Dienste über Profile freigeben

```
$ sudo ufw allow OpenSSH
$ sudo ufw allow "Apache Full"
$ sudo ufw status verbose
```

Beispiel 5.3: Ein eigenes Anwendungsprofil anlegen

```
$ sudo nano /etc/ufw/applications.d/meineanwendung
```

Beispiel 5.4: Inhalt des eigenen Profils

```
[Meineanwendung]
title=Eigene Testanwendung
description=Beispielprofil für Kapitel 5
ports=8080/tcp
```

Beispiel 5.5: UFW-Logging aktivieren und auswerten

```
$ sudo ufw logging medium
$ sudo tail -f /var/log/ufw.log
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Aktiviere ein neues Anwendungsprofil immer erst, nachdem du geprüft hast, dass der zugehörige Port tatsächlich korrekt und minimal gewählt ist. Ein Profil, das versehentlich einen größeren Portbereich öffnet als nötig, untergräbt das Prinzip der geringsten Rechte ebenso wie eine zu breite sudoers-Regel aus Kapitel 2.

UFW-Logging in der Stufe `full` erzeugt sehr viel Datenvolumen und eignet sich eher für eine kurze, gezielte Fehlersuche als für den Dauerbetrieb; `medium` ist für die meisten Zwecke ausreichend.

6. Praktische Laborübungen

Übung A: Mehrere Dienste über Profile freigeben

Aufgabenstellung: Richte UFW für SSH und einen zusätzlichen Testdienst mit Profilen ein. **Ausgangssituation:** `server1`, UFW noch nicht aktiv (oder nftables zuvor deaktiviert). **Voraussetzungen:** sudo-Rechte, zweite offene Sitzung. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 5.2. **Erwartetes Ergebnis:** `ufw status verbose` zeigt beide Profile als erlaubt. **Überprüfung:** SSH-Verbindung funktioniert weiterhin. **Fehleranalyse:** Profil nicht gefunden? Mit `ufw app list` die exakte Schreibweise prüfen. **Musterlösung:** `ufw status verbose` listet „OpenSSH“ und „Apache Full“ jeweils als „ALLOW“.

Übung B: Ein eigenes Profil anlegen und protokollieren

Aufgabenstellung: Lege das Profil aus Beispiel 5.3/5.4 an, aktiviere es, und beobachte das Logging. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 5.3-5.5; danach einen Testzugriff auf Port 8080 simulieren (z. B. mit `nc` oder `curl`). **Erwartetes Ergebnis:** Das neue Profil erscheint in `ufw app list`, ein Zugriffsversuch wird im Log sichtbar. **Überprüfung:** `/var/log/ufw.log` enthält einen passenden Eintrag. **Fehleranalyse:** Profil nicht erkannt? Syntax der Profildatei gegen Beispiel 5.4 prüfen. **Musterlösung:** Nach korrekter Einrichtung erscheint sowohl die Freigabe in `ufw status` als auch der zugehörige Logeintrag.

7. Typische Fehler

Fehler	Ursache	Behebung
Falscher Profilname verwendet	Schreibweise nicht geprüft	immer zuerst `ufw app list` konsultieren
Logging dauerhaft auf „full“ belassen	Testkonfiguration nie zurückgesetzt	nach der Fehlersuche auf „medium“ oder „low“ zurückstellen
Eigenes Profil mit zu großem Portbereich	Bequemlichkeit statt Prüfung	nur die tatsächlich benötigten Ports eintragen

8. Zusammenfassung

> **Zusammenfassung**
>
> - Anwendungsprofile bündeln Portregeln für bekannte und eigene Dienste unter sprechenden Namen.
> - Eigene Profile lassen sich einfach unter `/etc/ufw/applications.d/` ergänzen.
> - UFW-Logging in verschiedenen Stufen unterstützt gezielte Fehlersuche.
> - Jedes Profil sollte minimal und geprüft konfiguriert sein.

9. Kontrollfragen

1. Wozu dienen UFW-Anwendungsprofile? (leicht)
2. Welcher Befehl zeigt verfügbare Profile an? (leicht)
3. Wo liegen eigene UFW-Profile? (mittel)
4. Welche Logging-Stufe eignet sich für den Dauerbetrieb, welche für kurze Fehlersuche? (mittel)
5. Ein eigenes Anwendungsprofil wurde mit einem zu großen Portbereich angelegt und bereits produktiv genutzt. Wie gehst du vor, um das sicher zu korrigieren? (schwer)

Lösungshinweise: 1. Sie fassen Port- und Protokollregeln für einen Dienst unter einem sprechenden Namen zusammen. 2. `sudo ufw app list`. 3. Unter `/etc/ufw/applications.d/`. 4. „medium“ für den Dauerbetrieb, „full“ nur für kurze, gezielte Fehlersuche. 5. Zunächst den tatsächlich benötigten, minimalen Portbereich ermitteln, die Profildatei entsprechend anpassen, die Regel mit `ufw reload` neu laden und den Zugriff danach erneut testen, bevor die alte, zu weite Regel endgültig entfernt wird.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] mehrere Dienste über UFW-Profile freigegeben hast,
- [ ] ein eigenes Anwendungsprofil angelegt hast,
- [ ] UFW-Logging aktiviert und ausgewertet hast,
- [ ] ein zu weit gefasstes Profil sicher korrigieren kannst.

Kapitel 6: AppArmor und Mandatory Access Control

Lernziele

- Du erklärst den Unterschied zwischen Discretionary und Mandatory Access Control.
- Du prüfst den Status vorhandener AppArmor-Profile.
- Du setzt ein Profil von „complain“ auf „enforce“ und beobachtest die Wirkung.

1. Einführung

Alle bisherigen Rechtemechanismen (Band 1, Kapitel 3) folgen dem klassischen Unix-Modell: Der Eigentümer einer Ressource entscheidet über ihre Freigabe. **AppArmor** ergänzt dieses Modell um eine zusätzliche, vom Eigentümer unabhängige Schicht.

2. Technische Grundlagen

Das klassische Unix-Rechtemodell ist eine Form von **Discretionary Access Control** (DAC): Der Eigentümer einer Datei entscheidet selbst über deren Freigabe. **Mandatory Access Control** (MAC) legt dagegen systemweite Regeln fest, die unabhängig vom Eigentümerwillen gelten – ein Programm darf dann nur das, was sein Profil ausdrücklich erlaubt, selbst wenn der Dateieigentümer theoretisch mehr Zugriff gewähren würde.

AppArmor ist seit Debian 10 standardmäßig aktiviert und bindet seine Regeln an **Dateipfade** von Programmen, nicht an abstrakte Sicherheitskennungen wie das alternative SELinux.

Beispiel 6.1: Status von AppArmor prüfen

```
$ sudo aa-status
```

Die Ausgabe zeigt geladene Profile in den Modi **enforce** (Regeln werden aktiv durchgesetzt) und **complain** (Verstöße werden nur protokolliert, aber nicht verhindert).

3. Fachbegriffe

Ein **Profil** definiert für ein bestimmtes Programm, auf welche Dateien, Netzwerkfunktionen und Fähigkeiten es zugreifen darf. Der **complain-Modus** eignet sich zum Testen neuer Profile, ohne laufende Dienste sofort einzuschränken. **aa-genprof** und **aa-logprof** unterstützen dabei, ein Profil aus beobachtetem, tatsächlichem Programmverhalten halbautomatisch zu erstellen.

4. Praxis und Konfiguration

Beispiel 6.2: Ein Profil in den complain-Modus versetzen

```
$ sudo aa-complain /usr/sbin/nginx
```

Beispiel 6.3: Verstöße im complain-Modus beobachten

```
$ sudo journalctl -k | grep apparmor | grep ALLOWED
```

Beispiel 6.4: Profil nach erfolgreicher Testphase durchsetzen

```
$ sudo aa-enforce /usr/sbin/nginx
$ sudo aa-status
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Setze ein Profil niemals direkt auf „enforce“, ohne es vorher im complain-Modus beobachtet zu haben. Ein zu restriktives, ungetestetes Profil kann einen produktiven Dienst an eigentlich legitimen Aktionen hindern und damit die Verfügbarkeit (CIA-Triade, Band 1) beeinträchtigen.

MAC-Systeme wie AppArmor sind eine wirksame Ergänzung gegen Schwachstellen in einzelnen Programmen: Selbst wenn ein Angreifer über eine Lücke im Webserver Code ausführen kann, begrenzt ein korrektes AppArmor-Profil, was dieser Code anschließend tatsächlich tun darf.

6. Praktische Laborübungen

Übung A: Vorhandene Profile prüfen

Aufgabenstellung: Dokumentiere alle auf `server1` geladenen AppArmor-Profile und ihren jeweiligen Modus. **Ausgangssituation:** `server1`, Standard-AppArmor-Installation. **Voraussetzungen:** SSH-Zugriff. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 6.1. **Erwartetes Ergebnis:** Eine Liste geladener Profile, unterteilt nach „enforce“ und „complain“. **Überprüfung:** Die Summe beider Listen entspricht der Gesamtzahl geladener Profile. **Fehleranalyse:** `aa-status` nicht gefunden? Paket `apparmor-utils` nachinstallieren. **Musterlösung:** Eine vollständige Dokumentation nennt für jedes Profil Programmpfad und Modus.

Übung B: Ein Profil testen und durchsetzen

Aufgabenstellung: Installiere einen einfachen Testdienst (z. B. nginx), versetze sein Profil in den complain-Modus, erzeuge Verkehr, und setze es anschließend durch. **Ausgangssituation:** Ergebnis aus Übung A, nginx installiert. **Voraussetzungen:** sudo-Rechte. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 6.2-6.4. **Erwartetes Ergebnis:** Im complain-Modus werden Zugriffe protokolliert, aber nicht blockiert; nach `aa-enforce` werden tatsächliche Regelverstöße aktiv verhindert. **Überprüfung:** `aa-status` zeigt nginx nun unter „enforce“. **Fehleranalyse:** Dienst funktioniert nach `aa-enforce` nicht mehr? Protokollierte, im complain-Modus beobachtete, aber im Profil fehlende Zugriffe mit `aa-logprof` nachtragen. **Musterlösung:** nginx läuft nach dem Wechsel auf „enforce“ weiterhin normal, weil alle im Testbetrieb beobachteten, legitimen Zugriffe bereits im Profil erfasst wurden.

7. Typische Fehler

Fehler	Ursache	Behebung
Profil direkt auf „enforce“ gesetzt	Testphase übersprungen	immer zuerst „complain“, dann beobachten, dann „enforce“
Dienst nach Durchsetzung gestört	Profil deckt nicht alle legitimen Zugriffe ab	`aa-logprof` zur Nachpflege des Profils nutzen
MAC mit DAC verwechselt	Begriffe klingen ähnlich	DAC = Eigentümer entscheidet, MAC = systemweite, feste Regeln

8. Zusammenfassung

> **Zusammenfassung**
>
> - AppArmor ergänzt das klassische DAC-Modell um systemweite, pfadbasierte MAC-Regeln.
> - Profile laufen im testenden „complain“- oder durchsetzenden „enforce“-Modus.
> - Neue oder geänderte Profile sollten immer zuerst im complain-Modus beobachtet werden.
> - AppArmor begrenzt den Schaden einer Schwachstelle im jeweiligen Programm zusätzlich zu dessen eigenen Rechten.

9. Kontrollfragen

1. Was unterscheidet DAC von MAC? (leicht)
2. Welcher Befehl zeigt den Status aller AppArmor-Profile? (leicht)
3. Wozu dient der complain-Modus? (mittel)
4. Woran bindet AppArmor seine Profile? (mittel)
5. Nach dem Umstellen eines Profils auf „enforce“ funktioniert ein Dienst teilweise nicht mehr. Wie gehst du systematisch vor, um das Profil zu korrigieren, ohne es komplett neu zu erstellen? (schwer)

Lösungshinweise: 1. DAC lässt den Eigentümer über Freigaben entscheiden, MAC erzwingt systemweite, vom Eigentümer unabhängige Regeln. 2. `sudo aa-status`. 3. Er protokolliert Zugriffsverstöße, ohne sie zu blockieren, und eignet sich so zum Testen neuer Profile. 4. An Dateipfade der jeweiligen Programme. 5. Das Profil testweise zurück in den complain-Modus versetzen, den fehlenden Vorgang reproduzieren, die protokollierten, fehlenden Zugriffe mit `aa-logprof` in das bestehende Profil übernehmen, und erst danach erneut auf „enforce“ setzen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] den Unterschied zwischen DAC und MAC erklären kannst,
- [ ] den Status vorhandener AppArmor-Profile ausgelesen hast,
- [ ] ein Profil erfolgreich von „complain“ auf „enforce“ umgestellt hast,
- [ ] ein gestörtes Profil mit `aa-logprof` nachpflegen kannst.

Kapitel 7: Fail2ban und Angriffsschutz

Lernziele

- Du erklärst das Grundprinzip von Fail2ban (Filter, Jail, Aktion).
- Du aktivierst und konfigurierst ein Jail für SSH.
- Du erstellst ein eigenes Jail für einen zusätzlichen Dienst.

1. Einführung

Band 1 hat `pam_faillock` für lokale Anmeldeversuche eingeführt. **Fail2ban** erweitert dieses Prinzip auf beliebige Protokolldateien und Dienste und reagiert automatisiert mit einer Firewall-Sperre statt nur einer Kontosperrung.

2. Technische Grundlagen

Fail2ban beobachtet laufend Protokolldateien (Band 1, Kapitel 9) nach definierten Mustern (**Filtern**) und löst bei Überschreiten einer Schwelle eine **Aktion** aus, meist eine temporäre Firewall-Sperre der auffälligen Quelladresse. Die Kombination aus Filter, Schwellenwert und Aktion für einen bestimmten Dienst heißt **Jail**.

Beispiel 7.1: Fail2ban installieren

```
$ sudo apt install fail2ban
```

Fail2ban nutzt standardmäßig die bereits in Kapitel 4 vertiefte nftables-Integration oder iptables, um Sperren umzusetzen, ohne die eigentliche Firewall-Grundkonfiguration zu verändern.

3. Fachbegriffe

Ein **Jail** bündelt Filter, Protokollquelle, Schwellenwert und Aktion für einen konkreten Dienst. Ein **Filter** ist ein regulärer Ausdruck, der ein Muster in einer Protokollzeile erkennt (z. B. eine fehlgeschlagene SSH-Anmeldung). **bantime**, **findtime** und **maxretry** steuern gemeinsam, wie lange eine Sperre gilt, in welchem Zeitfenster Versuche gezählt werden, und ab wie vielen Versuchen gesperrt wird.

4. Praxis und Konfiguration

Beispiel 7.2: Lokale Konfiguration anlegen (niemals die Originaldatei direkt ändern)

```
$ sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
$ sudo nano /etc/fail2ban/jail.local
```

Beispiel 7.3: SSH-Jail konfigurieren

```
[sshd]
enabled = true
port = ssh
filter = sshd
logpath = /var/log/auth.log
maxretry = 5
findtime = 600
bantime = 1800
```

Beispiel 7.4: Dienst starten und Status prüfen

```
$ sudo systemctl enable --now fail2ban
$ sudo fail2ban-client status sshd
```

Beispiel 7.5: Eine Sperre manuell aufheben

```
$ sudo fail2ban-client set sshd unbanip 192.168.56.10
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Teste Fail2ban-Konfigurationen zunächst mit einer dir selbst bekannten Testadresse. Eine zu niedrige `maxretry`-Schwelle oder ein zu langes `bantime` kann dazu führen, dass du dich selbst versehentlich aussperrst – gerade im Übungslabor mit nur zwei Systemen ein reales Risiko.

Fail2ban ersetzt keine der bisherigen Maßnahmen (SSH-Härtung, Firewall, PAM), sondern ergänzt sie um eine automatisierte, musterbasierte Reaktion auf wiederholte Fehlversuche – ein weiteres Beispiel für Defense in Depth (Kapitel 1).

6. Praktische Laborübungen

Übung A: SSH-Jail einrichten und auslösen

Aufgabenstellung: Richte das Jail aus Beispiel 7.2/7.3 ein und löse eine Sperrung durch absichtliche Fehlversuche aus. **Ausgangssituation:** `server1` mit aktivem SSH-Dienst. **Voraussetzungen:** sudo-Rechte, Kenntnis der eigenen, zum Entsperren nötigen Adresse. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 7.2-7.4, danach fünf absichtliche Fehlversuche von `security-client`. **Erwartetes Ergebnis:** `fail2ban-client status sshd` zeigt die Adresse von `security-client` als gesperrt. **Überprüfung:** Eine neue SSH-Verbindung von dieser Adresse schlägt fehl, auch mit korrektem Passwort oder Schlüssel. **Fehleranalyse:** Keine Sperrung? Prüfe mit `fail2ban-client status sshd`, ob das Jail tatsächlich aktiv ist, und ob `logpath` auf die korrekte Datei zeigt. **Musterlösung:** Nach erfolgreicher Sperrung mit `fail2ban-client set sshd unbanip` die eigene Adresse wieder freigeben (Beispiel 7.5).

Übung B: Ein eigenes Jail für einen zusätzlichen Dienst

Aufgabenstellung: Erstelle ein Jail für einen zweiten Dienst deiner Wahl (z. B. einen einfachen Webserver-Login) nach demselben Muster wie das SSH-Jail. **Ausgangssituation:** Ergebnis aus Übung A, zusätzlicher Dienst installiert. **Voraussetzungen:** ein passender, auswertbarer Protokollpfad für den gewählten Dienst. **Schritt-für-Schritt-Anleitung:** passenden Filter in `/etc/fail2ban/filter.d/` prüfen oder anlegen, Jail analog zu Beispiel 7.3 in `jail.local` ergänzen. **Erwartetes Ergebnis:** Ein funktionierendes, zweites Jail parallel zum SSH-Jail. **Überprüfung:** `fail2ban-client status` listet beide Jails. **Fehleranalyse:** Filter erkennt keine Treffer? Regulären Ausdruck gegen eine echte Beispielzeile aus dem Protokoll testen. **Musterlösung:** Beide Jails laufen unabhängig voneinander und sperren jeweils nur bei tatsächlich zugehörigen Fehlversuchen.

7. Typische Fehler

Fehler	Ursache	Behebung
Eigene Adresse versehentlich gesperrt	zu niedrige Schwelle beim Testen	immer mit bekannter Entsperrmethode (Konsole, zweite Adresse) testen
Originaldatei `jail.conf` direkt verändert	Unkenntnis der Update-Konvention	immer `jail.local` für eigene Anpassungen nutzen
Filter erkennt Protokollformat nicht	falsches oder verändertes Logformat angenommen	Filter gegen echte Protokollzeilen testen

8. Zusammenfassung

Zusammenfassung

- Fail2ban kombiniert Filter, Protokollquelle und Aktion in einem Jail pro Dienst.
- `maxretry`, `findtime` und `bantime` steuern gemeinsam das Sperrverhalten.
- Eigene Anpassungen gehören immer in `jail.local`, nie in die Originaldatei.
- Fail2ban ergänzt, ersetzt aber nicht die übrigen Sicherheitsmaßnahmen.

9. Kontrollfragen

1. Was ist ein Jail in Fail2ban? (leicht)
2. In welcher Datei sollten eigene Anpassungen vorgenommen werden? (leicht)
3. Was bewirken `maxretry`, `findtime` und `bantime` zusammen? (mittel)
4. Warum ersetzt Fail2ban nicht die SSH-Härtung aus Kapitel 3? (mittel)
5. Du testest ein neues Jail im eigenen Zwei-Maschinen-Labor und sperrst dabei versehentlich die einzige verbliebene Zugangsadresse. Wie kommst du wieder rein? (schwer)

Lösungshinweise: 1. Die Kombination aus Filter, Protokollquelle, Schwellenwert und Aktion für einen bestimmten Dienst. 2. `jail.local`. 3. Sie legen fest, wie viele Fehlversuche in welchem Zeitfenster zu einer wie lange geltenden Sperre führen. 4. Weil es nur auf wiederholte Fehlversuche reagiert, aber keine der zugrunde liegenden Konfigurationsschwächen behebt. 5. Über die Konsole der Virtualisierungssoftware lokal auf `server1` anmelden (nicht per SSH) und dort `sudo fail2ban-client set sshd unbanip <eigene-adresse>` ausführen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] ein funktionierendes SSH-Jail eingerichtet und ausgelöst hast,
- [ ] eine Sperre manuell aufheben kannst,
- [ ] ein eigenes Jail für einen zweiten Dienst erstellt hast,
- [ ] erklären kannst, warum Fail2ban andere Maßnahmen ergänzt statt ersetzt.

Kapitel 8: Systemdienste und Prozesssicherheit

Lernziele

- Du deaktivierst nicht benötigte Systemdienste systematisch.
- Du härtest einen systemd-Dienst mit Sandboxing-Direktiven.
- Du begrenzt Ressourcen eines Dienstes gegen Erschöpfungsangriffe.

1. Einführung

Band 1 hat systemd-Units als Konzept eingeführt. Dieses Kapitel nutzt dieselbe Technik gezielt zur Härtung: nicht benötigte Dienste abschalten, benötigte Dienste einschränken.

2. Technische Grundlagen

systemd bietet eine Reihe von **Sandboxing-Direktiven**, die einem Dienst gezielt Fähigkeiten entziehen, unabhängig vom Dateisystem-Rechtemodell: Zugriff auf bestimmte Verzeichnisse verweigern, das Setzen neuer Capabilities verbieten, oder dem Dienst einen eigenen, isolierten temporären Bereich zuweisen.

Beispiel 8.1: Nicht benötigte Dienste identifizieren

```
$ systemctl list-unit-files --state=enabled
```

3. Fachbegriffe

ProtectSystem und **ProtectHome** beschränken den Schreibzugriff eines Dienstes auf Systemverzeichnisse bzw. Home-Verzeichnisse. **NoNewPrivileges** verhindert, dass ein Prozess während seiner Laufzeit zusätzliche Rechte erlangt, selbst über ein eigentlich zulässiges SUID-Programm. **PrivateTmp** gibt einem Dienst ein eigenes, von anderen Prozessen isoliertes `/tmp`-Verzeichnis.

4. Praxis und Konfiguration

Beispiel 8.2: Einen nicht benötigten Dienst deaktivieren

```
$ sudo systemctl disable --now avahi-daemon.service
```

Beispiel 8.3: Sandboxing-Direktiven in einer Service-Unit ergänzen

```
[Service]
ExecStart=/usr/sbin/nginx -g "daemon off;"
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true
NoNewPrivileges=true
```

Beispiel 8.4: Ressourcenbegrenzung gegen Erschöpfungsangriffe

```
[Service]
MemoryMax=512M
TasksMax=100
```

Beispiel 8.5: Wirkung der Sandboxing-Einstellungen prüfen

```
$ systemd-analyze security nginx.service
```

`systemd-analyze security` bewertet die Härtung einer Unit automatisiert und schlägt zusätzliche, noch nicht genutzte Direktiven vor.

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> `ProtectSystem=strict` und ähnliche Direktiven können einen Dienst daran hindern, Dateien zu schreiben, die er tatsächlich benötigt (z. B. eigene Protokolle oder Caches). Teste jede Sandboxing-Änderung gründlich, bevor sie produktiv übernommen wird, und ergänze bei Bedarf gezielte Ausnahmen über `ReadWritePaths`.

Ressourcenbegrenzungen wie `MemoryMax` und `TasksMax` schützen vor einem einzelnen, fehlerhaften oder angegriffenen Dienst, der durch übermäßigen Ressourcenverbrauch das gesamte System beeinträchtigt – ein Schutz für die Verfügbarkeit (CIA-Triade) anderer, auf demselben System laufender Dienste.

6. Praktische Laborübungen

Übung A: Nicht benötigte Dienste systematisch deaktivieren

Aufgabenstellung: Identifiziere und deaktiviere alle auf `server1` aktiven, aber nicht benötigten Dienste. **Ausgangssituation:** `server1` nach den vorherigen Kapiteln. **Voraussetzungen:** sudo-Rechte. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 8.1, 8.2. **Erwartetes Ergebnis:** Eine reduzierte Liste aktiver Dienste, von denen jeder begründet gebraucht wird. **Überprüfung:** `systemctl list-unit-files --state=enabled` zeigt nach der Bereinigung nur noch begründete Einträge. **Fehleranalyse:** Dienst nach Deaktivierung fehlt an anderer Stelle? Mit `systemctl enable --now` gezielt wieder aktivieren. **Musterlösung:** Eine vollständige Lösung dokumentiert jeden deaktivierten Dienst mit Begründung.

Übung B: Einen Dienst sandboxen und bewerten

Aufgabenstellung: Härte einen installierten Testdienst (z. B. nginx) mit den Direktiven aus Beispiel 8.3/8.4 und bewerte ihn mit `systemd-analyze security`. **Ausgangssituation:** nginx installiert und lauffähig. **Voraussetzungen:** sudo-Rechte. **Schritt-für-Schritt-Anleitung:** Override-Datei mit `systemctl edit nginx.service` anlegen, Direktiven ergänzen, Dienst neu starten. **Erwartetes Ergebnis:** Der Dienst läuft weiterhin normal, `systemd-analyze security nginx.service` zeigt einen verbesserten Bewertungswert gegenüber dem ungehärteten Zustand. **Überprüfung:** Vergleiche den Bewertungswert vor und nach der Änderung. **Fehleranalyse:** Dienst startet nicht mehr? Prüfe mit `journalctl -u nginx.service`, welche Direktive den Start verhindert, und lockere sie gezielt. **Musterlösung:** Eine erfolgreiche Lösung zeigt einen messbar verbesserten Sicherheitswert bei weiterhin funktionierendem Dienst.

7. Typische Fehler

Fehler	Ursache	Behebung
Dienst nach Sandboxing funktionsunfähig	zu strikte Direktiven ohne Ausnahmen	`ReadWritePaths` für tatsächlich benötigte Verzeichnisse ergänzen
Benötigter Dienst versehentlich deaktiviert	fehlende Dokumentation der Abhängigkeiten	vor jeder Deaktivierung Abhängigkeiten mit `systemctl list-dependencies` prüfen
Ressourcenbegrenzung zu knapp bemessen	kein realistischer Lasttest vorab	Grenzwerte gegen tatsächliche, erwartete Last austesten

8. Zusammenfassung

> **Zusammenfassung**
>
> - Nicht benötigte Dienste sollten systematisch identifiziert und deaktiviert werden.
> - systemd-Sandboxing-Direktiven schränken einen Dienst unabhängig vom Dateisystem-Rechtemodell ein.
> - `systemd-analyze security` bewertet die Härtung einer Unit automatisiert.
> - Ressourcenbegrenzungen schützen die Verfügbarkeit anderer Dienste auf demselben System.

9. Kontrollfragen

1. Welcher Befehl zeigt alle aktivierten systemd-Dienste? (leicht)
2. Was bewirkt `NoNewPrivileges`? (leicht)
3. Wozu dient `systemd-analyze security`? (mittel)
4. Warum kann `ProtectSystem=strict` einen Dienst stören? (mittel)
5. Ein Dienst benötigt Schreibzugriff auf genau ein Verzeichnis, das durch `ProtectSystem=strict` blockiert wird. Wie löst du das, ohne die gesamte Direktive zu entfernen? (schwer)

Lösungshinweise: 1. `systemctl list-unit-files --state=enabled`. 2. Es verhindert, dass ein Prozess während seiner Laufzeit zusätzliche Rechte erlangt. 3. Es bewertet automatisiert, wie gut eine Service-Unit gehärtet ist, und schlägt weitere Maßnahmen vor. 4. Weil es den Schreibzugriff auf weite Teile des Dateisystems verbietet, auch dort, wo ein Dienst legitim schreiben müsste. 5. Mit der Direktive `ReadWritePaths=/genau/dieses/verzeichnis` gezielt eine Ausnahme für das eine, benötigte Verzeichnis ergänzen, statt `ProtectSystem` vollständig zu entfernen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] nicht benötigte Dienste systematisch deaktiviert hast,
- [ ] einen Dienst mit Sandboxing-Direktiven gehärtet hast, ohne ihn zu beschädigen,
- [ ] `systemd-analyze security` zur Bewertung genutzt hast,
- [ ] eine gezielte Ausnahme mit `ReadWritePaths` setzen kannst.

Kapitel 9: Kernel-Sicherheit

Lernziele

- Du liest und setzt sicherheitsrelevante Kernel-Parameter über sysctl.
- Du deaktivierst nicht benötigte Kernelmodule gezielt.
- Du aktivierst grundlegenden Netzwerk-Schutz gegen gängige IP-Spoofing-Muster.

1. Einführung

Der Kernel selbst bringt zahlreiche, oft unterschätzte Sicherheitseinstellungen mit, die weit unterhalb der bisher behandelten Anwendungs- und Dienstebene wirken. Dieses Kapitel macht diese Ebene sichtbar und nutzbar.

2. Technische Grundlagen

sysctl liest und setzt Kernel-Parameter zur Laufzeit, ohne einen Neustart zu erfordern. Viele dieser Parameter betreffen Netzwerkverhalten und lassen sich direkt mit den in Band 1 (hping3-Bezüge) und diesem Buch behandelten Angriffsmustern verknüpfen.

Beispiel 9.1: Aktuelle sysctl-Werte auslesen

```
$ sysctl net.ipv4.tcp_syncookies
$ sysctl net.ipv4.conf.all.rp_filter
```

3. Fachbegriffe

SYN Cookies sind eine Kernel-Gegenmaßnahme gegen SYN-Flood-Angriffe, die keine Ressourcen für halb offene Verbindungen reserviert, sondern den nötigen Zustand kryptografisch in der Antwort kodiert. **Reverse Path Filtering** (`rp_filter`) verwirft Pakete, deren Absenderadresse laut Routing-Tabelle nicht über die tatsächlich genutzte Schnittstelle erreichbar sein dürfte – ein Grundschutz gegen bestimmte Spoofing-Muster.

4. Praxis und Konfiguration

Beispiel 9.2: Sicherheitsrelevante Parameter dauerhaft setzen (/etc/sysctl.d/99-hardening.conf)

```
net.ipv4.tcp_syncookies = 1
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.all.accept_source_route = 0
net.ipv4.icmp_echo_ignore_broadcasts = 1
kernel.randomize_va_space = 2
```

Parameter	Wirkung
`tcp_syncookies`	Schutz gegen SYN-Flood (vergleiche hping3-bezogene Angriffsmuster)
`rp_filter`	verwirft Pakete mit unplausibler Absenderadresse
`accept_source_route`	deaktiviert vom Absender vorgegebene Routingpfade

Parameter	Wirkung
`icmp_echo_ignore_broadcasts`	verhindert Mitwirkung an bestimmten, älteren Verstärkungsangriffen
`randomize_va_space`	aktiviert Speicherlayout-Randomisierung (ASLR)

Beispiel 9.3: Konfiguration anwenden

```
$ sudo sysctl -p /etc/sysctl.d/99-hardening.conf
```

Beispiel 9.4: Nicht benötigtes Kernelmodul dauerhaft deaktivieren

```
$ echo "install cramfs /bin/false" | sudo tee /etc/modprobe.d/disable-cramfs.conf
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Setze `net.ipv4.ip_forward` niemals versehentlich auf 1, sofern `server1` nicht ausdrücklich als Router fungieren soll – ein aktiviertes IP-Forwarding auf einem eigentlich isolierten Server kann unbeabsichtigt Netzwerkverkehr zwischen Segmenten durchlassen.

Kernel-Härtung wirkt systemweit und betrifft jeden Dienst gleichermaßen, im Unterschied zu den dienstspezifischen Maßnahmen aus Kapitel 8. Beide Ebenen ergänzen sich im Sinne von Defense in Depth.

6. Praktische Laborübungen

Übung A: Sicherheitsrelevante sysctl-Parameter setzen

Aufgabenstellung: Richte die Parameter aus Beispiel 9.2 dauerhaft ein. **Ausgangssituation:** `server1`, Standardwerte. **Voraussetzungen:** sudo-Rechte. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 9.2, 9.3. **Erwartetes Ergebnis:** `sysctl net.ipv4.tcp_syncookies` zeigt den Wert 1. **Überprüfung:** Ein Neustart von `server1` bestätigt, dass die Werte dauerhaft (nicht nur bis zum nächsten Neustart) gesetzt bleiben. **Fehleranalyse:** Werte nach Neustart zurückgesetzt? Prüfe, ob die Datei tatsächlich unter `/etc/sysctl.d/` liegt und korrekt benannt ist. **Musterlösung:** Alle fünf Parameter aus Beispiel 9.2 zeigen nach einem Neustart weiterhin die gehärteten Werte.

Übung B: Ein Kernelmodul gezielt deaktivieren

Aufgabenstellung: Identifiziere ein nicht benötigtes Kernelmodul und deaktiviere es dauerhaft. **Ausgangssituation:** `server1` mit Standard-Modulauswahl. **Voraussetzungen:** sudo-Rechte. **Schritt-für-Schritt-Anleitung:** `lsmod` auf nicht benötigte Dateisystemmodule prüfen (z. B. `cramfs`, `freevxfs`, falls geladen), dann Beispiel 9.4 anwenden. **Erwartetes Ergebnis:** Das Modul lässt sich nach einem Neustart nicht mehr laden. **Überprüfung:** `modprobe cramfs` meldet einen Fehler statt das Modul zu laden. **Fehleranalyse:** Modul lädt trotzdem? Prüfe die exakte Syntax der `modprobe.d`-Datei. **Musterlösung:** `modprobe cramfs` schlägt nach korrekter Konfiguration mit einer Fehlermeldung fehl.

7. Typische Fehler

Fehler	Ursache	Behebung
sysctl-Wert nur temporär gesetzt	direkt mit `sysctl -w` statt über Datei gesetzt	immer über `/etc/sysctl.d/` dauerhaft konfigurieren
IP-Forwarding versehentlich aktiviert	Kopiervorlage ungeprüft übernommen	jeden übernommenen Parameter einzeln auf Zweck prüfen
Benötigtes Modul deaktiviert	fehlende Prüfung der tatsächlichen Nutzung	vor der Deaktivierung mit `lsmod` und Dokumentation prüfen

8. Zusammenfassung

Zusammenfassung

- sysctl liest und setzt Kernel-Parameter zur Laufzeit, dauerhaft über Dateien unter `/etc/sysctl.d/`.
- SYN Cookies, Reverse Path Filtering und deaktiviertes Source Routing gehören zu den wichtigsten Netzwerk-Härtungsparametern.
- Nicht benötigte Kernelmodule lassen sich über `modprobe.d` dauerhaft blockieren.
- Kernel-Härtung wirkt systemweit und ergänzt dienstspezifische Maßnahmen.

9. Kontrollfragen

1. Welches Werkzeug liest und setzt Kernel-Parameter zur Laufzeit? (leicht)
2. Wogegen schützen SYN Cookies? (leicht)
3. Was bewirkt Reverse Path Filtering? (mittel)
4. Warum sollten sysctl-Änderungen über Dateien statt direkt mit `-w` gesetzt werden? (mittel)
5. `server1` soll niemals als Router fungieren, aber ein Kollege schlägt vor, `net.ipv4.ip_forward=1` „zur Sicherheit“ zu setzen, falls später doch Routing gebraucht wird. Wie bewertest du diesen Vorschlag? (schwer)

Lösungshinweise: 1. sysctl. 2. Gegen SYN-Flood-Angriffe, indem keine Ressourcen für halb offene Verbindungen reserviert werden. 3. Es verwirft Pakete, deren Absenderadresse laut Routing-Tabelle nicht plausibel über die genutzte Schnittstelle erreichbar wäre. 4. Weil direkt mit `-w` gesetzte Werte nach einem Neustart verloren gehen. 5. Das widerspricht dem Prinzip der geringsten Rechte: Eine Funktion „auf Vorrat“ zu aktivieren vergrößert die Angriffsfläche ohne aktuellen Nutzen; besser ist, sie erst bei tatsächlichem Bedarf gezielt zu aktivieren.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] sicherheitsrelevante sysctl-Parameter dauerhaft gesetzt hast,
- [ ] ein nicht benötigtes Kernelmodul dauerhaft deaktiviert hast,
- [ ] erklären kannst, wie SYN Cookies und Reverse Path Filtering wirken,
- [ ] eine „auf Vorrat“ aktivierte Funktion kritisch hinterfragen kannst.

Kapitel 10: Auditierung mit auditd

Lernziele

- Du erklärst den Unterschied zwischen auditd und der Protokollierung aus Band 1.
- Du richtest Audit-Regeln für sicherheitsrelevante Dateien und Syscalls ein.
- Du wertest Audit-Protokolle gezielt mit ausearch und aureport aus.

1. Einführung

Band 1 hat Logging als Mittel zur allgemeinen Nachvollziehbarkeit eingeführt. **auditd** geht gezielter vor: Es protokolliert nicht beliebige Anwendungsereignisse, sondern exakt definierte, sicherheitsrelevante Systemaufrufe und Dateizugriffe auf Kernelebene.

2. Technische Grundlagen

Der **Linux Audit Framework** hakt sich direkt in den Kernel ein und protokolliert Systemaufrufe (Syscalls, Band 1, Kapitel 2) sowie Zugriffe auf konkret überwachte Dateien, unabhängig davon, über welchen Weg oder welches Programm der Zugriff erfolgt – ein wesentlicher Unterschied zu anwendungsspezifischen Protokollen wie `auth.log`.

Beispiel 10.1: auditd installieren

```
$ sudo apt install auditd audispd-plugins
```

3. Fachbegriffe

Eine **Audit-Regel** definiert, welche Datei oder welcher Syscall überwacht werden soll, und mit welchem Schlüssel (`-k`) die resultierenden Ereignisse später wiederauffindbar markiert werden. **ausearch** durchsucht Audit-Protokolle gezielt nach einem solchen Schlüssel oder anderen Kriterien; **aureport** fasst sie zu lesbaren Übersichtsberichten zusammen.

4. Praxis und Konfiguration

Beispiel 10.2: Eine Datei-Zugriffsregel anlegen

```
$ sudo auditctl -w /etc/passwd -p wa -k passwd_aenderungen
```

Option	Bedeutung
`-w`	zu überwachende Datei
`-p wa`	überwacht Schreib- (write) und Attributänderungen (attribute)
`-k`	Schlüssel zur späteren, gezielten Suche

Beispiel 10.3: Regel dauerhaft machen (/etc/audit/rules.d/hardening.rules)

```
-w /etc/passwd -p wa -k passwd_aenderungen
-w /etc/shadow -p wa -k shadow_aenderungen
-w /etc/sudoers -p wa -k sudoers_aenderungen
```

Beispiel 10.4: Regeln laden und Protokoll auswerten

```
$ sudo augenrules --load
$ sudo ausearch -k passwd_aenderungen
$ sudo aureport -f
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Zu viele, zu breit gefasste Audit-Regeln (etwa die Überwachung ganzer Verzeichnisbäume mit hoher Schreibaktivität) erzeugen ein kaum noch auswertbares Datenvolumen und können die Systemleistung spürbar beeinträchtigen. Beschränke Regeln gezielt auf tatsächlich sicherheitsrelevante Dateien und Syscalls.

Audit-Protokolle sind, wie alle Protokolle (Band 1, Kapitel 9), nur so wertvoll wie ihre tatsächliche Auswertung. `auditd` liefert dafür deutlich präzisere, kernelnahe Daten als allgemeine Systemprotokolle – die Verantwortung, diese Daten regelmäßig zu prüfen, bleibt jedoch bestehen.

6. Praktische Laborübungen

Übung A: Audit-Regeln für kritische Dateien einrichten

Aufgabenstellung: Richte die Regeln aus Beispiel 10.3 ein und löse jeweils ein Ereignis aus. **Ausgangssituation:** `server1`, auditd installiert. **Voraussetzungen:** sudo-Rechte. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 10.3, 10.4; danach testweise `sudo visudo` öffnen und eine harmlose Kommentarzeile speichern. **Erwartetes Ergebnis:** `ausearch -k sudoers_aenderungen` zeigt den Zugriff mit Zeitstempel und ausführendem Benutzer. **Überprüfung:** Der Eintrag enthält den korrekten Benutzernamen und Zeitpunkt. **Fehleranalyse:** Kein Treffer? Prüfe mit `auditctl -l`, ob die Regel tatsächlich geladen ist. **Musterlösung:** Jede der drei Regeln liefert bei einem entsprechenden Zugriff einen nachvollziehbaren, mit dem jeweiligen Schlüssel auffindbaren Eintrag.

Übung B: Einen Übersichtsbericht erzeugen

Aufgabenstellung: Erzeuge mit `aureport` einen zusammenfassenden Bericht über alle Datei-Änderungsereignisse eines Testzeitraums. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 10.4 (`aureport -f`); zusätzlich `aureport -k` für eine Übersicht nach Schlüsseln. **Erwartetes Ergebnis:** Ein lesbarer Bericht mit Anzahl und Art der erfassten Ereignisse. **Überprüfung:** Die Anzahl der Ereignisse im Bericht passt zu den tatsächlich in Übung A ausgelösten Zugriffen. **Fehleranalyse:** Bericht leer? Prüfe den betrachteten Zeitraum mit `aureport --start` und `--end`. **Musterlösung:** Eine vollständige Lösung zeigt den Bericht und ordnet jeden Eintrag einem konkreten, in Übung A ausgelösten Ereignis zu.

7. Typische Fehler

Fehler	Ursache	Behebung
Regeln nach Neustart verschwunden	nur mit `auditctl` temporär gesetzt	Regeln dauerhaft unter `/etc/audit/rules.d/` ablegen
Zu viele Regeln, unauswertbares Datenvolumen	breite statt gezielte Überwachung	Regeln auf konkret sicherheitsrelevante Dateien beschränken
Audit-Ereignisse nie geprüft	wie bei jedem Logging ohne Auswertungsroutine	regelmäßige `aureport`-Durchsicht einplanen

8. Zusammenfassung

Zusammenfassung

- auditd protokolliert kernelnah, gezielt definierte Datei- und Syscall-Ereignisse.
- Audit-Regeln werden mit Schlüsseln versehen, um sie später gezielt wiederzufinden.
- `ausearch` sucht gezielt, `aureport` fasst zusammen.
- Regeln sollten gezielt und begrenzt bleiben, um auswertbar zu sein.

9. Kontrollfragen

1. Was überwacht auditd im Unterschied zu allgemeinen Systemprotokollen? (leicht)
2. Wozu dient der `-k`-Parameter einer Audit-Regel? (leicht)
3. Was ist der Unterschied zwischen `ausearch` und `aureport`? (mittel)
4. Warum sollten Audit-Regeln nicht zu breit gefasst werden? (mittel)
5. Du sollst nachträglich prüfen, ob in den letzten 30 Tagen jemand `/etc/sudoers` verändert hat, aber es existiert noch keine Audit-Regel dafür. Was ist die Konsequenz, und wie gehst du für die Zukunft vor? (schwer)

Lösungshinweise: 1. Kernelnahe Systemaufrufe und konkret definierte Dateizugriffe, unabhängig vom auslösenden Programm. 2. Er markiert die Ereignisse für eine spätere, gezielte Suche. 3. `ausearch` sucht einzelne Ereignisse, `aureport` fasst sie zu lesbaren Übersichten zusammen. 4. Weil zu viele Regeln ein kaum auswertbares Datenvolumen erzeugen und die Leistung beeinträchtigen können. 5. Ohne vorab eingerichtete Regel lässt sich die Vergangenheit nicht rückwirkend auditieren; die Regel muss jetzt eingerichtet werden (Beispiel 10.2/10.3), damit zukünftige Änderungen nachvollziehbar sind – ein Beleg dafür, Audit-Regeln präventiv statt reaktiv einzurichten.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] Audit-Regeln für mehrere kritische Dateien eingerichtet hast,
- [ ] ein ausgelöstes Ereignis mit `ausearch` gezielt gefunden hast,
- [ ] einen zusammenfassenden Bericht mit `aureport` erzeugt hast,
- [ ] erklären kannst, warum Audit-Regeln präventiv eingerichtet werden müssen.

Kapitel 11: Integritätsüberwachung mit AIDE

Lernziele

- Du erklärst den Zweck der Integritätsüberwachung gegenüber der Auditierung aus Kapitel 10.
- Du initialisierst eine AIDE-Datenbank und führst einen Vergleichslauf durch.
- Du passt den Überwachungsumfang gezielt an.

1. Einführung

auditd (Kapitel 10) protokolliert Ereignisse, während sie geschehen. **AIDE** (Advanced Intrusion Detection Environment) verfolgt einen anderen Ansatz: Es vergleicht den aktuellen Zustand des Dateisystems mit einer zuvor erstellten, vertrauenswürdigen Referenz und meldet jede Abweichung.

2. Technische Grundlagen

AIDE erstellt eine **Datenbank** mit kryptografischen Prüfsummen und Metadaten (Rechte, Eigentümer, Zeitstempel) für ausgewählte Dateien und Verzeichnisse. Ein späterer Vergleichslauf berechnet dieselben Werte erneut und meldet jede Datei, die hinzugekommen, verändert oder entfernt wurde.

Beispiel 11.1: AIDE installieren und Datenbank initialisieren

```
$ sudo apt install aide
$ sudo aideinit
```

`aideinit` erzeugt die initiale Referenzdatenbank; dieser Vorgang sollte auf einem bekanntermaßen unveränderten, vertrauenswürdigen System erfolgen.

3. Fachbegriffe

Die **Referenzdatenbank** ist der gespeicherte Vertrauensanker, gegen den jeder spätere Lauf verglichen wird. Ein **Vergleichslauf** (Check) berechnet den aktuellen Zustand neu und vergleicht ihn mit der Referenz. Integritätsüberwachung ergänzt auditd um die Fähigkeit, auch Veränderungen zu erkennen, die ohne vorab eingerichtete Audit-Regel geschahen (vergleiche Kapitel 10, Kontrollfrage 5).

4. Praxis und Konfiguration

Beispiel 11.2: Überwachten Umfang konfigurieren (/etc/aide/aide.conf, Auszug)

```
/etc    CONTENT_EX
/bin    CONTENT_EX
/sbin   CONTENT_EX
!/var/log
```

Das Ausrufezeichen schließt ein Verzeichnis ausdrücklich von der Überwachung aus – sinnvoll für Verzeichnisse mit erwartbar häufigen, legitimen Änderungen wie Protokolle.

Beispiel 11.3: Einen Vergleichslauf durchführen

```
$ sudo aide --check
```

Beispiel 11.4: Datenbank nach legitimer Änderung aktualisieren

```
$ sudo aideinit
$ sudo cp /var/lib/aide/aide.db.new /var/lib/aide/aide.db
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Erstelle die initiale AIDE-Datenbank ausschließlich auf einem System, dessen Unversehrtheit zu diesem Zeitpunkt sichergestellt ist. Eine Referenzdatenbank, die auf einem bereits kompromittierten System erstellt wird, bestätigt dem Angreifer versehentlich seinen eigenen, bereits erfolgten Eingriff als „normal“.

Nach jeder legitimen, beabsichtigten Systemänderung (ein Update, eine neue Konfiguration) muss die Referenzdatenbank aktualisiert werden – sonst meldet jeder folgende Vergleichslauf diese legitime Änderung fälschlich als Abweichung, was mit der Zeit zu ignorierten, „gewohnten“ Warnmeldungen führt und die eigentliche Schutzwirkung untergräbt.

6. Praktische Laborübungen

Übung A: AIDE einrichten und einen sauberen Vergleichslauf durchführen

Aufgabenstellung: Initialisiere AIDE auf `server1` und führe einen ersten Vergleichslauf ohne vorherige Änderungen durch. **Ausgangssituation:** `server1` im aktuellen, als vertrauenswürdig geltenden Zustand. **Voraussetzungen:** sudo-Rechte. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 11.1, 11.3. **Erwartetes Ergebnis:** Der erste Vergleichslauf direkt nach der Initialisierung meldet keine Abweichungen. **Überprüfung:** Die Ausgabe von `aide --check` zeigt „No differences found“ oder vergleichbar. **Fehleranalyse:** Unerwartete Abweichungen trotz keiner Änderung? Prüfe, ob zwischen Initialisierung und Check automatisierte Prozesse (z. B. Log-Rotation) gelaufen sind, die überwachte Verzeichnisse betreffen. **Musterlösung:** Ein sauberer erster Lauf ohne gemeldete Abweichungen bestätigt eine korrekt erstellte Referenzdatenbank.

Übung B: Eine Abweichung erkennen und die Datenbank aktualisieren

Aufgabenstellung: Verändere testweise eine überwachte Datei, erkenne die Abweichung, und aktualisiere danach die Datenbank bewusst. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** Eine Testzeile an `/etc/motd` oder eine ähnliche, überwachte Datei anhängen, danach `aide --check` erneut ausführen, danach Beispiel 11.4 anwenden. **Erwartetes Ergebnis:** Der erste Check nach der Änderung meldet die Datei als verändert; nach der Aktualisierung der Datenbank meldet ein erneuter Check keine Abweichung mehr. **Überprüfung:** Der gemeldete Dateipfad stimmt mit der tatsächlich veränderten Datei überein. **Fehleranalyse:** Keine Meldung? Prüfe, ob die veränderte Datei tatsächlich im überwachten Bereich aus Beispiel 11.2 liegt. **Musterlösung:** Vor der Aktualisierung meldet AIDE die Abweichung korrekt, danach ist sie als neuer, legitimer Zustand übernommen.

7. Typische Fehler

Fehler	Ursache	Behebung
Referenzdatenbank auf unsicherem System erstellt	Integrität des Ausgangszustands nicht sichergestellt	Datenbank nur auf einem geprüft vertrauenswürdigen System initialisieren
Datenbank nach legitimen Änderungen nie aktualisiert	fehlende Routine	Aktualisierung fest in den Änderungsprozess einplanen
Zu viele Verzeichnisse mit natürlicher Änderungsrate überwacht	Konfiguration ungeprüft übernommen	häufig legitim wechselnde Bereiche gezielt ausschließen

8. Zusammenfassung

Zusammenfassung

- AIDE vergleicht den aktuellen Dateisystemzustand mit einer zuvor erstellten Referenzdatenbank.
- Die Referenzdatenbank muss auf einem geprüft unversehrten System erstellt werden.
- Nach jeder legitimen Änderung muss die Datenbank aktualisiert werden, sonst verlieren Warnungen ihre Aussagekraft.
- AIDE ergänzt auditd um die Fähigkeit, auch nicht vorab geplante Änderungen zu erkennen.

9. Kontrollfragen

1. Was vergleicht AIDE bei einem Check? (leicht)
2. Welcher Befehl initialisiert die AIDE-Datenbank? (leicht)
3. Warum muss die initiale Datenbank auf einem vertrauenswürdigen System erstellt werden? (mittel)
4. Was passiert, wenn die Datenbank nach legitimen Änderungen nie aktualisiert wird? (mittel)
5. Ein Team ignoriert AIDE-Warnungen zunehmend, „weil sowieso ständig etwas gemeldet wird“. Was ist die wahrscheinliche Ursache, und wie behebst du sie? (schwer)

Lösungshinweise: 1. Kryptografische Prüfsummen und Metadaten des aktuellen Zustands gegen die gespeicherte Referenzdatenbank. 2. `aideinit`. 3. Weil sonst ein bereits vorhandener, unbemerkter Eingriff fälschlich als normaler Ausgangszustand gespeichert würde. 4. Jede legitime Änderung wird fälschlich als Abweichung gemeldet, was zu ignorierten, „gewohnten“ Warnungen führt. 5. Wahrscheinlich wurde die Datenbank nach legitimen Änderungen nicht aktualisiert, oder der überwachte Umfang ist zu breit gefasst; Abhilfe schaffen eine konsequente Aktualisierungsroutine nach jeder beabsichtigten Änderung und eine Eingrenzung des überwachten Bereichs auf tatsächlich seltene Änderungen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] eine AIDE-Datenbank erfolgreich initialisiert hast,
- [ ] eine Dateiveränderung mit einem Vergleichslauf erkannt hast,
- [ ] die Datenbank nach einer legitimen Änderung korrekt aktualisiert hast,
- [ ] erklären kannst, warum ignorierte Integritätswarnungen ein eigenständiges Risiko darstellen.

Kapitel 12: Malware-Erkennung mit ClamAV

Lernziele

- Du ordnest die Rolle von Virenscannern unter Linux realistisch ein.
- Du richtest ClamAV mit aktuellen Signaturen ein und führst einen Scan durch.
- Du automatisierst regelmäßige Scans und die Signaturaktualisierung.

1. Einführung

Virenscanner werden mit Windows-Systemen assoziiert, spielen aber auch unter Linux eine Rolle, besonders auf Servern, die Dateien für andere Systeme bereitstellen (Dateiserver, E-Mail-Gateways). Dieses Kapitel ordnet ClamAV realistisch ein und zeigt die praktische Einrichtung.

2. Technische Grundlagen

ClamAV ist ein quelloffener Virenscanner, der Dateien gegen eine Datenbank bekannter Schadsoftware-Signaturen prüft. Diese Signaturdatenbank muss regelmäßig aktualisiert werden, da sie nur bereits bekannte Schadsoftware erkennt – ein grundsätzliches Prinzip, das Virenscanner mit der signaturbasierten Intrusion Detection (Band 3) teilen.

Beispiel 12.1: ClamAV installieren

```
$ sudo apt install clamav clamav-daemon
$ sudo systemctl stop clamav-freshclam
$ sudo freshclam
$ sudo systemctl start clamav-freshclam
```

3. Fachbegriffe

freshclam ist der Dienst, der die Signaturdatenbank von ClamAV automatisch aktualisiert. Ein **On-Demand-Scan** prüft gezielt einen angegebenen Pfad bei Bedarf, ein **On-Access-Scan** prüft Dateien laufend beim Zugriff – unter Linux seltener im Dauerbetrieb genutzt, da er spürbare Leistung kostet.

4. Praxis und Konfiguration

Beispiel 12.2: Einen gezielten Scan durchführen

```
$ sudo clamscan -r /home --log=/var/log/clamav/scan.log
```

Beispiel 12.3: Regelmäßigen Scan per Cron einrichten

```
$ sudo crontab -e
```

Beispiel 12.4: Cron-Eintrag für wöchentlichen Scan

```
0 3 * * 0 /usr/bin/clamscan -r /home --log=/var/log/clamav/weekly.log
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> ClamAV erkennt ausschließlich bereits bekannte Schadsoftware-Muster. Es ersetzt keine der bisherigen Maßnahmen dieses Buches und erkennt insbesondere keine gezielt gegen ein bestimmtes System entwickelte, bisher unbekannte Schadsoftware oder einen manuell durchgeführten Angriff über eine Fehlkonfiguration.

Der praktische Nutzen eines Virenscanners unter Linux liegt oft weniger im Schutz des Linux-Systems selbst als im Schutz der Systeme, die darüber Dateien beziehen: Ein Dateiserver, der Windows-Clients bedient, kann mit ClamAV verhindern, bereits bekannte, für Windows schädliche Dateien ungeprüft weiterzureichen.

6. Praktische Laborübungen

Übung A: ClamAV einrichten und einen Testscan durchführen

Aufgabenstellung: Richte ClamAV ein und scanne ein Testverzeichnis mit der bekannten EICAR-Testdatei. **Ausgangssituation:** `server1`, ClamAV noch nicht installiert. **Voraussetzungen:** sudo-Rechte, Internetzugang für die initiale Signaturaktualisierung. **Schritt-für-Schritt-Anleitung:**

Befehle zu Übung A

```
# EICAR-Testdatei ist eine standardisierte, harmlose Testdatei zur
# Überprüfung von Virenscannern, öffentlich von eicar.org verfügbar
$ clamscan /pfad/zur/eicar-testdatei
```

Erwartetes Ergebnis: ClamAV erkennt die EICAR-Testdatei als „Eicar-Test-Signature FOUND“. **Überprüfung:** Die Erkennung erfolgt zuverlässig und ohne Fehlalarm bei anderen, harmlosen Dateien im selben Verzeichnis. **Fehleranalyse:** Keine Erkennung? Prüfe mit `freshclam`, ob die Signaturdatenbank aktuell ist. **Musterlösung:** Die Ausgabe von `clamscan` zeigt die EICAR-Datei eindeutig als erkannt, alle anderen Testdateien als „OK“.

Übung B: Automatisierten, regelmäßigen Scan einrichten

Aufgabenstellung: Richte einen wöchentlichen, automatisierten Scan nach Beispiel 12.3/12.4 ein. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 12.3, 12.4. **Erwartetes Ergebnis:** Ein Cron-Eintrag, der den Scan wöchentlich um 3 Uhr nachts ausführt und protokolliert. **Überprüfung:** Ein manuell mit `clamscan` nachgestellter Testlauf erzeugt dieselbe Protokolldatei wie der geplante Cron-Eintrag. **Fehleranalyse:** Cron-Job läuft nicht? Prüfe mit `grep CRON /var/log/syslog`, ob der Job zum geplanten Zeitpunkt tatsächlich gestartet wurde. **Musterlösung:** Die Protokolldatei unter `/var/log/clamav/weekly.log` enthält nach dem geplanten Zeitpunkt einen neuen, vollständigen Scanbericht.

7. Typische Fehler

Fehler	Ursache	Behebung
Virenscanner als vollständiger Schutz missverstanden	Unkenntnis der Grenzen signaturbasierter Erkennung	ClamAV als eine von mehreren Schichten (Defense in Depth) einordnen
Signaturdatenbank veraltet	`freshclam` nicht aktiv oder nie geprüft	Dienststatus regelmäßig kontrollieren
Scan belastet produktiven Betrieb spürbar	Scan zu Stoßzeiten eingeplant	Scanzeitpunkt in Zeiten geringer Last legen

8. Zusammenfassung

Zusammenfassung

- ClamAV erkennt bereits bekannte Schadsoftware anhand einer regelmäßig aktualisierten Signaturdatenbank.
- Es erkennt keine unbekannte, gezielt entwickelte Schadsoftware oder Konfigurationsfehler.
- Der praktische Nutzen liegt oft im Schutz nachgelagerter, andersartiger Systeme.
- Regelmäßige, automatisierte Scans außerhalb von Lastspitzen sind der sinnvolle Regelbetrieb.

9. Kontrollfragen

1. Wonach sucht ClamAV bei einem Scan? (leicht)
2. Welcher Dienst aktualisiert die Signaturdatenbank? (leicht)
3. Was ist die EICAR-Testdatei? (mittel)
4. Warum schützt ClamAV nicht vor gezielt entwickelter, unbekannter Schadsoftware? (mittel)
5. Ein Linux-Dateiserver bedient ausschließlich andere Linux-Clients. Lohnt sich ClamAV in diesem Szenario überhaupt, und warum? (schwer)

Lösungshinweise: 1. Nach Übereinstimmungen mit bekannten Schadsoftware-Signaturen. 2. `freshclam`. 3. Eine standardisierte, harmlose Testdatei, mit der sich die Erkennungsfunktion eines Virenscanners gefahrlos prüfen lässt. 4. Weil die signaturbasierte Erkennung auf bereits bekannten Mustern beruht und neue, bisher unbekannte Schadsoftware per Definition nicht erfasst. 5. Der Nutzen ist geringer, aber nicht null: Linux-spezifische Schadsoftware existiert ebenfalls, und ein Scan kann versehentlich hochgeladene, für andere Systeme schädliche Dateien dennoch erkennen, falls der Server später auch für andere Plattformen genutzt wird.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] ClamAV eingerichtet und die Signaturdatenbank aktualisiert hast,
- [ ] einen Testscan mit der EICAR-Datei erfolgreich durchgeführt hast,
- [ ] einen automatisierten, regelmäßigen Scan eingerichtet hast,
- [ ] die realistischen Grenzen signaturbasierter Erkennung erklären kannst.

Kapitel 13: Backup- und Wiederherstellungskonzepte

Lernziele

- Du erklärst, warum Backups Teil des Sicherheitskonzepts sind, nicht nur der IT-Betriebsroutine.
- Du richtest ein automatisiertes, inkrementelles Backup ein.
- Du führst eine vollständige Testwiederherstellung durch.

1. Einführung

Jede bisher in diesem Buch behandelte Maßnahme kann versagen: eine unentdeckte Schwachstelle, ein menschlicher Fehler, ein Hardwareausfall. Ein funktionierendes Backup ist die letzte Verteidigungslinie – und zugleich die am häufigsten vernachlässigte, weil ihr Wert erst im Ernstfall sichtbar wird.

2. Technische Grundlagen

Ein **Vollbackup** sichert den gesamten Datenbestand, ein **inkrementelles Backup** nur die Änderungen seit dem letzten Backup (egal welcher Art), ein **differenzielles Backup** die Änderungen seit dem letzten Vollbackup. Inkrementelle Backups sind platzsparender, machen die Wiederherstellung aber von einer vollständigen, unbeschädigten Kette aller Zwischenschritte abhängig.

Beispiel 13.1: Backup-Strategien im Vergleich (Konzept)

```
Vollbackup:        Mo [████████] Di [████████] Mi [████████]
Inkrementell:      Mo [████████] Di [██] Mi [██]
Differenziell:     Mo [████████] Di [██] Mi [████]
```

3. Fachbegriffe

Die **3-2-1-Regel** empfiehlt mindestens drei Kopien der Daten, auf mindestens zwei unterschiedlichen Speichermedien, davon mindestens eine Kopie an einem anderen physischen Ort. Eine **Testwiederherstellung** (Restore Test) prüft, ob ein Backup tatsächlich nutzbar ist – ein nie getestetes Backup ist ein ungeprüftes Versprechen, kein verlässlicher Schutz.

4. Praxis und Konfiguration

Beispiel 13.2: Ein einfaches, inkrementelles Backup mit rsync

```
$ rsync -a --link-dest=/backup/aktuell /home/ /backup/$(date +%Y-%m-%d)/
$ rm -f /backup/aktuell
$ ln -s /backup/$(date +%Y-%m-%d) /backup/aktuell
```

`--link-dest` verweist auf unveränderte Dateien des vorherigen Backups per Hardlink, statt sie erneut zu kopieren – platzsparend und dennoch pro Zeitpunkt vollständig wiederherstellbar.

Beispiel 13.3: Backup automatisieren (Cron-Eintrag)

```
0 2 * * * /usr/local/bin/backup.sh
```

Beispiel 13.4: Eine Testwiederherstellung durchführen

```
$ mkdir /tmp/restore_test
$ rsync -a /backup/aktuell/ /tmp/restore_test/
$ diff -r /home /tmp/restore_test
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Ein Backup, das auf demselben System wie die Originaldaten liegt, schützt nicht gegen einen vollständigen Systemausfall, einen Hardware-Defekt oder eine Verschlüsselungsschadsoftware, die auch angeschlossene Laufwerke erfasst. Die 3-2-1-Regel ist deshalb keine akademische Empfehlung, sondern eine direkte Lehre aus realen Vorfällen.

Backups selbst sind ein Angriffsziel: Wer Schreibzugriff auf ein Backup-System erlangt, kann dort hinterlegte, vermeintlich sichere Kopien ebenso manipulieren wie die Originaldaten. Zugriffsrechte auf Backup-Speicher verdienen dieselbe Sorgfalt wie auf Produktivsysteme (Band 1, Kapitel 3).

6. Praktische Laborübungen

Übung A: Automatisiertes, inkrementelles Backup einrichten

Aufgabenstellung: Richte ein tägliches, inkrementelles Backup des Verzeichnisses `/home` nach Beispiel 13.2/13.3 ein. **Ausgangssituation:** `server1`, noch kein automatisiertes Backup. **Voraussetzungen:** ein separates Backup-Zielverzeichnis oder eine zweite virtuelle Festplatte. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 13.2, 13.3. **Erwartetes Ergebnis:** Nach dem ersten Lauf existiert ein vollständiges Backup; nach einer kleinen Teständerung und einem zweiten Lauf wird nur die Änderung neu gespeichert, unveränderte Dateien bleiben als Hardlinks erhalten. **Überprüfung:** `du -sh` auf beide Backup-Zeitpunkte zeigt, dass der zweite Durchlauf deutlich weniger zusätzlichen Speicherplatz benötigt. **Fehleranalyse:** Jeder Lauf kopiert alles neu? Prüfe, ob `--link-dest` korrekt auf das vorherige Backup verweist. **Musterlösung:** Zwei aufeinanderfolgende Backup-Verzeichnisse mit nahezu identischem, aber durch Hardlinks platzsparendem Inhalt.

Übung B: Eine vollständige Testwiederherstellung durchführen

Aufgabenstellung: Stelle das Backup aus Übung A in ein separates Testverzeichnis wieder her und vergleiche es mit dem Original. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** ausreichend freier Speicherplatz für die Testwiederherstellung. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 13.4. **Erwartetes Ergebnis:** `diff -r` zeigt keine inhaltlichen Unterschiede zwischen Original und wiederhergestellter Kopie. **Überprüfung:** Stichprobenartiges Öffnen einzelner wiederhergestellter Dateien bestätigt die vollständige Lesbarkeit. **Fehleranalyse:** Unterschiede gefunden? Prüfe, ob zwischen Backup-Erstellung und Vergleich noch Änderungen am Original vorgenommen wurden. **Musterlösung:** Eine erfolgreiche Testwiederherstellung zeigt keine Abweichungen und bestätigt damit die tatsächliche Nutzbarkeit des Backups.

7. Typische Fehler

Fehler	Ursache	Behebung
Backup nie getestet	Vertrauen ohne Überprüfung	regelmäßige Testwiederherstellungen fest einplanen
Backup auf demselben System wie Original	fehlende Beachtung der 3-2-1-Regel	externe oder räumlich getrennte Kopie einrichten
Backup-Zugriff genauso offen wie Produktivsystem	Rechtekonzept nicht auf Backup übertragen	Zugriffsrechte für Backup-Speicher eigenständig prüfen

8. Zusammenfassung

> **Zusammenfassung**
>
> - Inkrementelle Backups sparen Speicherplatz, machen die Wiederherstellung aber von der vollständigen Kette abhängig.
> - Die 3-2-1-Regel schützt gegen den Ausfall eines einzelnen Systems oder Speicherortes.
> - Ein nie getestetes Backup ist kein verlässlicher Schutz.
> - Backup-Speicher verdient dieselbe Zugriffssorgfalt wie das Produktivsystem.

9. Kontrollfragen

1. Was unterscheidet ein inkrementelles von einem differenziellen Backup? (leicht)
2. Was besagt die 3-2-1-Regel? (leicht)
3. Warum ist eine Testwiederherstellung notwendig, auch wenn das Backup „sicher gelaufen“ ist? (mittel)
4. Wie spart `--link-dest` bei rsync Speicherplatz? (mittel)
5. Ein Unternehmen hat tägliche Backups, aber noch nie eine Wiederherstellung getestet, und das einzige Backup-Ziel liegt im selben Serverraum. Welche zwei konkreten Risiken bestehen, und wie würdest du sie priorisieren? (schwer)

Lösungshinweise: 1. Inkrementell sichert Änderungen seit dem letzten beliebigen Backup, differenziell seit dem letzten Vollbackup. 2. Mindestens drei Kopien, auf zwei verschiedenen Medien, eine davon an einem anderen Ort. 3. Weil ein erfolgreicher Backup-Lauf nicht zwangsläufig ein tatsächlich nutzbares, vollständiges Backup bedeutet. 4. Es verweist auf unveränderte Dateien des vorherigen Backups per Hardlink, statt sie erneut zu kopieren. 5. Erstens ein ungetestetes Backup könnte im Ernstfall unbrauchbar sein, zweitens ein lokales Backup schützt nicht gegen einen Totalausfall des Serverraums (Brand, Diebstahl); die räumliche Trennung hat oft die höhere Priorität, da ein ungetestetes, aber vorhandenes Backup wenigstens eine Chance bietet, während ein bei einem Totalausfall gleichzeitig zerstörtes Backup gar keine.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] ein automatisiertes, inkrementelles Backup eingerichtet hast,
- [ ] eine vollständige Testwiederherstellung erfolgreich durchgeführt hast,

- [ ] die 3-2-1-Regel auf ein eigenes Szenario anwenden kannst,
- [ ] Backup-Speicher als eigenständig schützenswert einordnen kannst.

Kapitel 14: Docker- und Container-Sicherheit

Lernziele

- Du erklärst, warum Container-Isolation schwächer ist als die einer vollständigen virtuellen Maschine.
- Du betreibst einen Container mit eingeschränkten Rechten statt mit root.
- Du begrenzt Ressourcen und Capabilities eines Containers gezielt.

1. Einführung

Container haben die Art, wie Anwendungen betrieben werden, grundlegend verändert, bringen aber eigene, oft unterschätzte Sicherheitsfragen mit. Dieses Kapitel ordnet Container-Isolation realistisch ein und zeigt konkrete Härtungsschritte.

2. Technische Grundlagen

Container teilen sich den Kernel des Host-Systems und nutzen zur Isolation Kernel-Mechanismen wie **Namespaces** (trennen Sicht auf Prozesse, Netzwerk, Dateisystem) und **Control Groups** (cgroups, begrenzen Ressourcenverbrauch). Diese Isolation ist schwächer als die einer vollständigen virtuellen Maschine mit eigenem Kernel: Eine Schwachstelle im Host-Kernel kann theoretisch alle Container gleichzeitig betreffen (Container-Escape).

Beispiel 14.1: Docker installieren

```
$ sudo apt install docker.io
$ sudo systemctl enable --now docker
```

3. Fachbegriffe

Ein **Container-Escape** ist ein erfolgreicher Ausbruch aus der Container-Isolation auf den Host oder in andere Container. **Rootless Docker** betreibt den Docker-Daemon selbst ohne root-Rechte. Eine **Capability** (Band 3 dieser Buchreihe, dort vertieft) lässt sich auch einem Container gezielt entziehen oder hinzufügen, unabhängig von seinen übrigen Rechten.

4. Praxis und Konfiguration

Beispiel 14.2: Einen Container mit eingeschränktem Benutzer starten

```
$ docker run --user 1000:1000 --rm -it debian:bookworm bash
```

Beispiel 14.3: Ressourcen und Capabilities begrenzen

```
$ docker run --memory=256m --cpus=0.5 \
    --cap-drop=ALL --cap-add=NET_BIND_SERVICE \
    --read-only nginx
```

Option	Wirkung
`--memory`	begrenzt den maximalen Arbeitsspeicher
`--cpus`	begrenzt die maximale CPU-Nutzung
`--cap-drop=ALL`	entzieht alle Standard-Capabilities
`--cap-add=NET_BIND_SERVICE`	gibt gezielt nur die tatsächlich benötigte Capability zurück
`--read-only`	macht das Container-Dateisystem schreibgeschützt

Beispiel 14.4: Laufende Container und ihre Rechte prüfen

```
$ docker inspect <container-id> | grep -i cap
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Starte Container niemals mit `--privileged`, außer wenn ein konkreter, unvermeidbarer Grund vorliegt und dieser dokumentiert ist. Diese Option hebt praktisch die gesamte Container-Isolation auf und macht einen Container nahezu gleichwertig zum Host-System.

Container-Images aus nicht vertrauenswürdigen oder ungeprüften Quellen bergen dasselbe Risiko wie ungeprüfte Software-Pakete (Band 1, Kapitel 8): Ein manipuliertes Image kann Schadcode bereits vor dem ersten Start enthalten. Offizielle, signierte Images aus bekannten Registries sind deshalb vorzuziehen.

6. Praktische Laborübungen

Übung A: Einen Container mit eingeschränkten Rechten betreiben

Aufgabenstellung: Starte einen Container mit einem nicht-root-Benutzer und bestätige die eingeschränkten Rechte. **Ausgangssituation:** Docker installiert, ein Beispiel-Image verfügbar. **Voraussetzungen:** sudo-Rechte. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 14.2, danach im Container `id` und `touch /test` versuchen. **Erwartetes Ergebnis:** `id` zeigt die UID 1000 statt root, ein Schreibversuch in ein root-geschütztes Verzeichnis schlägt fehl. **Überprüfung:** Der Container läuft erfolgreich, aber mit sichtbar eingeschränkten Rechten. **Fehleranalyse:** Container läuft trotzdem als root? Prüfe, ob der Parameter `--user` korrekt übergeben wurde. **Musterlösung:** `id` im Container zeigt `uid=1000 gid=1000`, root-geschützte Aktionen schlagen fehl.

Übung B: Ressourcen und Capabilities begrenzen

Aufgabenstellung: Starte einen Container nach Beispiel 14.3 und bestätige die Begrenzungen. **Ausgangssituation:** wie Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 14.3, 14.4. **Erwartetes Ergebnis:** `docker inspect` zeigt nur die ausdrücklich hinzugefügte Capability, alle anderen sind entfernt; ein Schreibversuch außerhalb vorgesehener Volumes schlägt wegen `--read-only` fehl. **Überprüfung:** Der Container funktioniert weiterhin für seinen eigentlichen Zweck (z. B. nginx liefert weiterhin Inhalte aus). **Fehleranalyse:** Dienst startet nicht? Prüfe, ob eine tatsächlich benötigte Capability versehentlich mit entfernt wurde. **Musterlösung:** Der Dienst läuft mit minimalen Rechten weiter, `docker inspect` bestätigt die eng begrenzte Capability-Liste.

7. Typische Fehler

Fehler	Ursache	Behebung
Container standardmäßig als root betrieben	`--user` nicht gesetzt	Benutzer explizit in jedem Startbefehl oder im Dockerfile angeben
`--privileged` aus Bequemlichkeit genutzt	Fehlerursache nicht gezielt gesucht	genaue Ursache des Problems statt pauschaler Rechteerweiterung ermitteln
Images aus beliebigen Quellen genutzt	fehlende Prüfung der Herkunft	offizielle, signierte Images bevorzugen

8. Zusammenfassung

Zusammenfassung

- Container-Isolation über Namespaces und cgroups ist schwächer als eine vollständige virtuelle Maschine.
- Container sollten mit eingeschränktem Benutzer statt root betrieben werden.
- Ressourcenbegrenzung und gezielte Capability-Vergabe reduzieren das Schadenspotenzial deutlich.
- `--privileged` hebt die Isolation praktisch vollständig auf und sollte vermieden werden.

9. Kontrollfragen

1. Welche zwei Kernel-Mechanismen bilden die Grundlage der Container-Isolation? (leicht)
2. Was ist ein Container-Escape? (leicht)
3. Warum ist Container-Isolation schwächer als die einer vollständigen virtuellen Maschine? (mittel)
4. Was bewirkt `--cap-drop=ALL` in Kombination mit gezieltem `--cap-add`? (mittel)
5. Ein Entwicklerteam startet alle Container standardmäßig mit `--privileged`, „weil sonst manchmal Berechtigungsfehler auftreten“. Wie gehst du systematisch vor, um das zu korrigieren? (schwer)

Lösungshinweise: 1. Namespaces und Control Groups (cgroups). 2. Ein erfolgreicher Ausbruch aus der Container-Isolation auf den Host oder in andere Container. 3. Weil Container denselben Kernel wie der Host teilen, während eine virtuelle Maschine einen eigenen, vollständig getrennten Kernel besitzt. 4. Es entfernt zunächst alle Standard-Capabilities und vergibt anschließend nur die tatsächlich benötigten gezielt zurück. 5. Den konkreten Berechtigungsfehler für jeden betroffenen Container einzeln ermitteln (z. B. mit `docker logs`), die tatsächlich fehlende, einzelne Capability identifizieren, und diese gezielt mit `--cap-add` statt mit `--privileged` nachrüsten.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen Container mit eingeschränktem, nicht-root Benutzer betrieben hast,
- [ ] Ressourcen und Capabilities eines Containers gezielt begrenzt hast,
- [ ] erklären kannst, warum `--privileged` ein erhebliches Risiko darstellt,
- [ ] einen pauschalen Rechteerweiterungs-Workaround durch eine gezielte Lösung ersetzen kannst.

Kapitel 15: Webserver-Sicherheit

Lernziele

- Du konfigurierst grundlegende Sicherheits-Header für einen Webserver.
- Du richtest TLS mit einem gültigen Zertifikat ein.
- Du beschränkst den Webserver-Prozess nach dem Prinzip der geringsten Rechte.

1. Einführung

Ein Webserver ist oft der am stärksten exponierte Dienst eines Systems. Dieses Kapitel verbindet Themen aus mehreren vorherigen Kapiteln (TLS aus Band 1, Sandboxing aus Kapitel 8) gezielt am Beispiel eines Webservers.

2. Technische Grundlagen

HTTP-Antwort-Header können das Verhalten von Browsern gezielt steuern und bestimmte Angriffsklassen erschweren, ohne die Anwendung selbst zu verändern. **TLS** verschlüsselt die Verbindung (Band 1, Kapitel 10); ein gültiges Zertifikat bestätigt zusätzlich die Identität des Servers.

Beispiel 15.1: nginx installieren

```
$ sudo apt install nginx
```

3. Fachbegriffe

HSTS (HTTP Strict Transport Security) weist Browser an, eine Website zukünftig ausschließlich über HTTPS aufzurufen, selbst wenn ein Nutzer versehentlich HTTP eingibt. **Content-Security-Policy** (CSP) begrenzt, aus welchen Quellen eine Webseite Skripte, Stile oder andere Inhalte laden darf, und erschwert damit bestimmte XSS-Varianten (Ethical-Hacking-Handbuch der Schwesterreihe, dort vertieft). **Let's Encrypt** ist eine kostenlose, automatisierte Zertifizierungsstelle.

4. Praxis und Konfiguration

Beispiel 15.2: Sicherheits-Header in der nginx-Konfiguration ergänzen

```
add_header Strict-Transport-Security "max-age=63072000" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "DENY" always;
add_header Content-Security-Policy "default-src 'self'" always;
```

Header	Wirkung
`Strict-Transport-Security`	erzwingt zukünftig ausschließlich HTTPS
`X-Content-Type-Options`	verhindert, dass Browser Dateitypen abweichend vom angegebenen interpretieren
`X-Frame-Options`	verhindert Einbettung der Seite in fremde iframes (Clickjacking-Schutz)
`Content-Security-Policy`	begrenzt erlaubte Inhaltsquellen

Beispiel 15.3: Ein TLS-Zertifikat mit Let's Encrypt einrichten (benötigt öffentlich erreichbaren Server)

```
$ sudo apt install certbot python3-certbot-nginx
$ sudo certbot --nginx -d beispiel-domain.de
```

> **Gut zu wissen**
>
> Im isolierten Laborumfeld dieses Buches ist `server1` nicht öffentlich erreichbar, Let's Encrypt kann die Domain-Inhaberschaft deshalb nicht validieren. Für die Laborübung dieses Kapitels wird stattdessen ein selbstsigniertes Testzertifikat genutzt (Beispiel 15.4).

Beispiel 15.4: Ein selbstsigniertes Testzertifikat erzeugen

```
$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:4096 \
    -keyout /etc/ssl/private/test.key -out /etc/ssl/certs/test.crt
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Ein selbstsigniertes Zertifikat verschlüsselt die Verbindung zwar korrekt, bestätigt aber nicht die Identität des Servers gegenüber einem Browser und löst deshalb eine Warnmeldung aus. Für ein produktives System ist ein Zertifikat einer anerkannten Zertifizierungsstelle wie Let's Encrypt die richtige Wahl.

Die Kombination aus TLS, Sicherheits-Headern und einem nach Kapitel 8 sandboxed nginx-Prozess ist ein konkretes Beispiel für Defense in Depth: Jede Schicht schützt gegen eine andere Angriffsklasse, keine ersetzt die andere.

6. Praktische Laborübungen

Übung A: Sicherheits-Header einrichten und prüfen

Aufgabenstellung: Ergänze die Header aus Beispiel 15.2 und bestätige sie in der Antwort des Servers. **Ausgangssituation:** nginx installiert, Standardkonfiguration. **Voraussetzungen:** sudo-Rechte. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 15.2, danach Konfiguration testen und neu laden. **Erwartetes Ergebnis:** Eine Anfrage an den Server zeigt alle vier Header in der Antwort. **Überprüfung:**

Prüfbefehl zu Übung A

```
$ curl -I http://192.168.56.20
```

Die Ausgabe enthält alle vier konfigurierten Header. **Fehleranalyse:** Header fehlen? Prüfe, ob `nginx -t` die Konfiguration als fehlerfrei bestätigt und ob `systemctl reload nginx` tatsächlich erfolgte. **Musterlösung:** `curl -I` zeigt alle vier Sicherheits-Header korrekt gesetzt.

Übung B: TLS mit selbstsigniertem Zertifikat einrichten

Aufgabenstellung: Richte TLS mit dem Testzertifikat aus Beispiel 15.4 ein und bestätige die verschlüsselte Verbindung. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** Zertifikat erzeugen (Beispiel 15.4), in der nginx-Konfiguration als `ssl_certificate`/`ssl_certificate_key` einbinden, Port 443 in der Firewall freigeben (Kapitel 4/5). **Erwartetes Ergebnis:** Eine HTTPS-Verbindung funktioniert, der Browser oder `curl` meldet eine Zertifikatswarnung, die Verbindung selbst ist aber verschlüsselt. **Überprüfung:**

Prüfbefehl zu Übung B

```
$ curl -k -I https://192.168.56.20
```

Fehleranalyse: Verbindung schlägt fehl? Prüfe, ob Port 443 tatsächlich in der Firewall freigegeben wurde. **Musterlösung:** `curl -k` (ignoriert die erwartete Zertifikatswarnung) zeigt eine erfolgreiche, verschlüsselte HTTPS-Antwort.

7. Typische Fehler

Fehler	Ursache	Behebung
Sicherheits-Header nach Neukonfiguration nicht sichtbar	`reload` vergessen	nach jeder Änderung `systemctl reload nginx` ausführen
Selbstsigniertes Zertifikat produktiv eingesetzt	Verwechslung von Testlabor und Produktivbetrieb	für produktive Systeme ausschließlich anerkannte Zertifizierungsstellen nutzen
Port 443 nach TLS-Einrichtung vergessen freizugeben	Firewall-Anpassung übersehen	Firewall-Regeln nach jeder neuen Diensteinrichtung prüfen

8. Zusammenfassung

Zusammenfassung

- Sicherheits-Header wie HSTS, X-Frame-Options und CSP erschweren bestimmte Angriffsklassen ohne Codeänderung.
- TLS mit einem anerkannten Zertifikat verschlüsselt und bestätigt die Serveridentität zugleich.
- Selbstsignierte Zertifikate eignen sich für Labor und Test, nicht für den Produktivbetrieb.
- Die Kombination mehrerer Maßnahmen aus diesem und vorherigen Kapiteln ist ein konkretes Beispiel für Defense in Depth.

9. Kontrollfragen

1. Wozu dient der HSTS-Header? (leicht)
2. Was unterscheidet ein selbstsigniertes von einem durch eine Zertifizierungsstelle signierten Zertifikat? (leicht)
3. Wogegen schützt `X-Frame-Options`? (mittel)
4. Warum lässt sich Let's Encrypt im isolierten Laborumfeld dieses Buches nicht direkt nutzen? (mittel)
5. Ein Kunde möchte CSP so restriktiv wie möglich einsetzen, aber seine Webanwendung lädt Skripte von mehreren externen Diensten. Wie gehst du vor, um beides zu vereinbaren? (schwer)

Lösungshinweise: 1. Er erzwingt künftig ausschließlich HTTPS-Verbindungen zu dieser Seite. 2. Ein selbstsigniertes Zertifikat bestätigt die Serveridentität nicht gegenüber Dritten, ein signiertes schon. 3. Gegen Clickjacking, also das Einbetten der Seite in ein fremdes, täuschendes iframe. 4. Weil Let's Encrypt die öffentliche Erreichbarkeit der Domain prüft, die im isolierten Labor nicht gegeben ist. 5. Jede tatsächlich genutzte, externe Quelle einzeln und gezielt in die CSP aufnehmen (statt eines pauschal offenen `*`), sodass die Richtlinie weiterhin restriktiv bleibt, aber die legitim benötigten Quellen funktionieren.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] Sicherheits-Header erfolgreich eingerichtet und per `curl` bestätigt hast,
- [ ] TLS mit einem Testzertifikat eingerichtet hast,
- [ ] den Unterschied zwischen selbstsigniertem und anerkanntem Zertifikat erklären kannst,
- [ ] eine restriktive CSP mit tatsächlich benötigten externen Quellen vereinbaren kannst.

Kapitel 16: Sicherheitskonzepte für Unternehmensserver

Lernziele

- Du verbindest die Maßnahmen aus Kapitel 1-15 zu einem zusammenhängenden Sicherheitskonzept.
- Du erstellst eine Risikobewertung für einen konkreten Serverzweck.
- Du dokumentierst ein vollständiges Hardening-Konzept nach einheitlichem Schema.

1. Einführung

Die vorherigen fünfzehn Kapitel haben Einzelmaßnahmen behandelt. Ein Unternehmensserver braucht mehr: ein zusammenhängendes Konzept, das begründet, welche Maßnahmen warum in welcher Priorität umgesetzt werden, abgestimmt auf den konkreten Zweck des Servers.

2. Technische Grundlagen

Ein **Sicherheitskonzept** verbindet Bestandsaufnahme (Band 1, Kapitel 1), Risikobewertung, konkrete Maßnahmen und deren Dokumentation zu einem nachvollziehbaren Gesamtbild. Die **Risikobewertung** schätzt für jede identifizierte Bedrohung grob ab, wie wahrscheinlich sie ist und wie schwerwiegend ihre Auswirkung wäre, um Maßnahmen priorisieren zu können, statt alles gleichzeitig und gleich intensiv umzusetzen.

Beispiel 16.1: Eine einfache Risikomatrix (Konzept)

```
                 geringe Auswirkung    hohe Auswirkung
hohe W'keit      mittlere Priorität    höchste Priorität
geringe W'keit   niedrigste Priorität  mittlere Priorität
```

3. Fachbegriffe

Ein **Schutzbedarf** beschreibt, wie kritisch die Vertraulichkeit, Integrität und Verfügbarkeit eines konkreten Systems für die Organisation ist – ein interner Testserver hat typischerweise einen geringeren Schutzbedarf als ein Server mit Kundendaten. Die **Dokumentationspflicht** verlangt, jede getroffene Sicherheitsentscheidung so festzuhalten, dass sie auch für Dritte (etwa eine spätere Übernahme der Administration) nachvollziehbar bleibt.

4. Praxis und Konfiguration

Beispiel 16.2: Struktur eines Sicherheitskonzepts (Gliederungsvorschlag)

```
1. Zweck und Schutzbedarf des Systems
2. Bestandsaufnahme (Dienste, Daten, Schnittstellen)
3. Risikobewertung je identifizierter Bedrohung
4. Umgesetzte Maßnahmen, Kapitel für Kapitel zugeordnet
5. Verbleibende, bewusst akzeptierte Restrisiken
6. Verantwortlichkeiten und Aktualisierungsturnus
```

Beispiel 16.3: Eine Maßnahme einer Bedrohung zuordnen (Auszug)

```
Bedrohung: Brute-Force gegen SSH
Maßnahme:  Schlüsselauthentifizierung (Band 1, Kap. 5) + Fail2ban-Jail (Kap. 7)
Priorität: hoch (hohe Wahrscheinlichkeit, mittlere bis hohe Auswirkung)
Status:    umgesetzt am [Datum]
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Ein Sicherheitskonzept, das nie aktualisiert wird, veraltet ebenso wie ein nicht gepatchtes Paket (Band 1, Kapitel 8). Jede neue Bedrohung, jede neue Softwareversion und jede Änderung am Serverzweck sollte eine Überprüfung des bestehenden Konzepts auslösen.

Nicht jedes Risiko lässt sich oder muss vollständig beseitigt werden. Ein bewusst dokumentiertes **Restrisiko**, das mit Begründung akzeptiert wird, ist fachlich redlicher als ein unausgesprochenes, unbemerktes Risiko, das schlicht übersehen wurde.

6. Praktische Laborübungen

Übung A: Ein Sicherheitskonzept für server1 erstellen

Aufgabenstellung: Erstelle ein vollständiges, schriftliches Sicherheitskonzept für `server1` nach der Gliederung aus Beispiel 16.2. **Ausgangssituation:** `server1` nach allen vorherigen fünfzehn Kapiteln dieses Bandes. **Voraussetzungen:** Zugriff auf alle bisherigen Übungsdokumentationen dieses Bandes. **Schritt-für-Schritt-Anleitung:** Gliederung aus Beispiel 16.2 übernehmen, jeden Abschnitt mit den tatsächlichen Gegebenheiten von `server1` füllen. **Erwartetes Ergebnis:** Ein zusammenhängendes Dokument, das von der Zweckbestimmung bis zu Verantwortlichkeiten reicht. **Überprüfung:** Jede in den Kapiteln 1-15 umgesetzte Maßnahme taucht im Konzept mit Zuordnung zu einer konkreten Bedrohung auf. **Fehleranalyse:** Lücken im Konzept? Mit der Kapitelübersicht aus dem Vorwort dieses Bandes abgleichen, ob ein Themenbereich übersehen wurde. **Musterlösung:** Ein vollständiges Konzept verbindet alle sechzehn Kapitel zu einer in sich stimmigen, nachvollziehbaren Dokumentation.

Übung B: Eine Risikobewertung mit Priorisierung durchführen

Aufgabenstellung: Identifiziere fünf konkrete Bedrohungen für `server1` und ordne sie nach der Risikomatrix aus Beispiel 16.1 ein. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** keine zusätzlichen. **Schritt-für-Schritt-Anleitung:** Fünf Bedrohungen benennen (z. B. Brute-Force, Fehlkonfiguration, Datenträgerverlust, Container-Escape, Datenintegritätsverlust), jeweils Wahrscheinlichkeit und Auswirkung grob einschätzen und in die Matrix einordnen. **Erwartetes Ergebnis:** Eine nach Priorität sortierte Liste von fünf Bedrohungen mit zugeordneter Gegenmaßnahme aus den vorherigen Kapiteln. **Überprüfung:** Die höchste Priorität entspricht tatsächlich der am schwersten wiegenden, wahrscheinlichsten Bedrohung. **Fehleranalyse:** Einschätzungen wirken willkürlich? Mit einer zweiten Person gegenprüfen, ob die Einschätzung nachvollziehbar begründet werden kann. **Musterlösung:** Eine vollständige Lösung ordnet jeder der fünf Bedrohungen eine begründete Priorität und eine konkrete, bereits umgesetzte oder geplante Gegenmaßnahme zu.

7. Typische Fehler

Fehler	Ursache	Behebung
Konzept nur einmalig erstellt, nie aktualisiert	fehlender Aktualisierungsturnus	festen Überprüfungszeitpunkt im Konzept selbst festlegen
Alle Risiken gleich priorisiert	fehlende systematische Bewertung	Risikomatrix konsequent anwenden
Restrisiken unausgesprochen übersehen	fehlendes Bewusstsein für bewusste Akzeptanz	jedes nicht vollständig behobene Risiko ausdrücklich dokumentieren

8. Zusammenfassung

> **Zusammenfassung**
>
> - Ein Sicherheitskonzept verbindet Bestandsaufnahme, Risikobewertung, Maßnahmen und Dokumentation.
> - Eine Risikomatrix hilft, Maßnahmen nach Wahrscheinlichkeit und Auswirkung zu priorisieren.
> - Verbleibende Restrisiken sollten bewusst dokumentiert, nicht stillschweigend übersehen werden.
> - Ein Sicherheitskonzept braucht einen festen Aktualisierungsturnus, um nicht zu veralten.

9. Kontrollfragen

1. Was gehört in die Gliederung eines vollständigen Sicherheitskonzepts? (leicht)
2. Wonach priorisiert eine Risikomatrix Bedrohungen? (leicht)
3. Was ist ein Restrisiko? (mittel)
4. Warum ist ein unausgesprochenes Restrisiko problematischer als ein dokumentiertes? (mittel)
5. Zwei Server haben identische technische Konfiguration, einer verarbeitet interne Testdaten, der andere Kundendaten. Sollte ihr Sicherheitskonzept identisch sein? Begründe. (schwer)

Lösungshinweise: 1. Zweck, Bestandsaufnahme, Risikobewertung, Maßnahmen, Restrisiken, Verantwortlichkeiten. 2. Nach Eintrittswahrscheinlichkeit und Schwere der Auswirkung. 3. Ein nicht vollständig behobenes, aber bewusst akzeptiertes Risiko. 4. Weil ein dokumentiertes Restrisiko bewusst abgewogen wurde, während ein unausgesprochenes schlicht übersehen wurde und jederzeit unerwartet eintreten kann. 5. Nein, der unterschiedliche Schutzbedarf (interne Testdaten versus Kundendaten) rechtfertigt unterschiedliche Prioritäten und Schwellenwerte, selbst bei identischer technischer Ausgangskonfiguration – etwa strengere Zugriffsbeschränkungen oder häufigere Audits für den Server mit Kundendaten.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] ein vollständiges Sicherheitskonzept für einen Server erstellt hast,
- [ ] eine Risikobewertung mit Priorisierung durchgeführt hast,
- [ ] Restrisiken bewusst und nachvollziehbar dokumentiert hast,
- [ ] erklären kannst, warum identischer Schutzbedarf nicht automatisch aus identischer Technik folgt.

Abschlussprojekt Band 2: Professionelles Server-Hardening

Härte den aus Band 1 übernommenen Server vollständig nach den in diesem Band behandelten Verfahren:

1. **Systemanalyse:** Baseline nach Kapitel 1, SUID- und Rechte-Revision nach Kapitel 2.
2. **Hardening-Maßnahmen:** SSH-Hardening (Kapitel 3), vertiefte Firewall-Konfiguration (Kapitel 4 oder 5), Kernel-Härtung (Kapitel 9).
3. **AppArmor:** mindestens ein Dienst von „complain“ auf „enforce“ überführt (Kapitel 6).
4. **Fail2ban:** mindestens zwei Jails aktiv (Kapitel 7).
5. **Auditierung:** Audit-Regeln für kritische Dateien, Integritätsüberwachung mit AIDE (Kapitel 10-11).
6. **Sicherheitsbericht:** ein vollständiges Sicherheitskonzept nach dem Schema aus Kapitel 16, das alle umgesetzten Maßnahmen begründet zusammenfasst.

Dieses Projekt bildet die Grundlage für Band 3, in dem derselbe, nun gehärtete Server aus Angreiferperspektive analysiert wird.

Verzeichnis der Befehlsbeispiele

Beispiel 1.1: Der Hardening-Kreislauf (Konzept) … Seite 9
Beispiel 1.2: Eine einfache Baseline-Dokumentation beginnen … Seite 9
Beispiel 1.3: Einen Prüfpunkt manuell gegen eine Empfehlung abgleichen (Konzept) … Seite 10
Beispiel 2.1: Alle SUID-Programme eines Systems auflisten … Seite 12
Beispiel 2.2: Ein unnötiges SUID-Bit entfernen … Seite 12
Beispiel 2.3: Passwort-Ablauf für ein Konto konfigurieren … Seite 12
Beispiel 2.4: sudoers-Regel verschärfen (mit visudo) … Seite 13
Beispiel 3.1: Zugriff auf bestimmte Benutzer und Netzwerke beschränken … Seite 15
Beispiel 3.2: Moderne Algorithmen erzwingen (Auszug aus sshd_config) … Seite 15
Beispiel 3.3: Inaktive Verbindungen automatisch trennen … Seite 15
Beispiel 3.4: Konfiguration testen und übernehmen … Seite 16
Beispiel 4.1: Ein wiederverwendbares Set definieren … Seite 18
Beispiel 4.2: Verworfene Pakete protokollieren … Seite 19
Beispiel 4.3: Verbindungsrate auf SSH begrenzen … Seite 19
Beispiel 4.4: Protokollierte Verwürfe einsehen … Seite 19
Beispiel 5.1: Verfügbare Anwendungsprofile anzeigen … Seite 22
Beispiel 5.2: Mehrere Dienste über Profile freigeben … Seite 22
Beispiel 5.3: Ein eigenes Anwendungsprofil anlegen … Seite 22
Beispiel 5.4: Inhalt des eigenen Profils … Seite 22
Beispiel 5.5: UFW-Logging aktivieren und auswerten … Seite 23
Beispiel 6.1: Status von AppArmor prüfen … Seite 25
Beispiel 6.2: Ein Profil in den complain-Modus versetzen … Seite 25
Beispiel 6.3: Verstöße im complain-Modus beobachten … Seite 26
Beispiel 6.4: Profil nach erfolgreicher Testphase durchsetzen … Seite 26
Beispiel 7.1: Fail2ban installieren … Seite 28
Beispiel 7.2: Lokale Konfiguration anlegen (niemals die Originaldatei direkt ändern) … Seite 28
Beispiel 7.3: SSH-Jail konfigurieren … Seite 29
Beispiel 7.4: Dienst starten und Status prüfen … Seite 29
Beispiel 7.5: Eine Sperre manuell aufheben … Seite 29
Beispiel 8.1: Nicht benötigte Dienste identifizieren … Seite 32
Beispiel 8.2: Einen nicht benötigten Dienst deaktivieren … Seite 32
Beispiel 8.3: Sandboxing-Direktiven in einer Service-Unit ergänzen … Seite 32
Beispiel 8.4: Ressourcenbegrenzung gegen Erschöpfungsangriffe … Seite 33
Beispiel 8.5: Wirkung der Sandboxing-Einstellungen prüfen … Seite 33
Beispiel 9.1: Aktuelle sysctl-Werte auslesen … Seite 35
Beispiel 9.2: Sicherheitsrelevante Parameter dauerhaft setzen (/etc/sysctl.d/99-hardening.conf) … Seite 35
Beispiel 9.3: Konfiguration anwenden … Seite 36
Beispiel 9.4: Nicht benötigtes Kernelmodul dauerhaft deaktivieren … Seite 36
Beispiel 10.1: auditd installieren … Seite 38
Beispiel 10.2: Eine Datei-Zugriffsregel anlegen … Seite 38
Beispiel 10.3: Regel dauerhaft machen (/etc/audit/rules.d/hardening.rules) … Seite 39
Beispiel 10.4: Regeln laden und Protokoll auswerten … Seite 39
Beispiel 11.1: AIDE installieren und Datenbank initialisieren … Seite 41
Beispiel 11.2: Überwachten Umfang konfigurieren (/etc/aide/aide.conf, Auszug) … Seite 41
Beispiel 11.3: Einen Vergleichslauf durchführen … Seite 42
Beispiel 11.4: Datenbank nach legitimer Änderung aktualisieren … Seite 42
Beispiel 12.1: ClamAV installieren … Seite 44
Beispiel 12.2: Einen gezielten Scan durchführen … Seite 44
Beispiel 12.3: Regelmäßigen Scan per Cron einrichten … Seite 44
Beispiel 12.4: Cron-Eintrag für wöchentlichen Scan … Seite 45
Befehle zu Übung A … Seite 45
Beispiel 13.1: Backup-Strategien im Vergleich (Konzept) … Seite 47

Beispiel 13.2: Ein einfaches, inkrementelles Backup mit rsync … Seite 47
Beispiel 13.3: Backup automatisieren (Cron-Eintrag) … Seite 48
Beispiel 13.4: Eine Testwiederherstellung durchführen … Seite 48
Beispiel 14.1: Docker installieren … Seite 51
Beispiel 14.2: Einen Container mit eingeschränktem Benutzer starten … Seite 51
Beispiel 14.3: Ressourcen und Capabilities begrenzen … Seite 51
Beispiel 14.4: Laufende Container und ihre Rechte prüfen … Seite 52
Beispiel 15.1: nginx installieren … Seite 54
Beispiel 15.2: Sicherheits-Header in der nginx-Konfiguration ergänzen … Seite 54
Beispiel 15.3: Ein TLS-Zertifikat mit Let's Encrypt einrichten (benötigt öffentlich erreichbaren Server) … Seite 55
Beispiel 15.4: Ein selbstsigniertes Testzertifikat erzeugen … Seite 55
Prüfbefehl zu Übung A … Seite 55
Prüfbefehl zu Übung B … Seite 56
Beispiel 16.1: Eine einfache Risikomatrix (Konzept) … Seite 58
Beispiel 16.2: Struktur eines Sicherheitskonzepts (Gliederungsvorschlag) … Seite 58
Beispiel 16.3: Eine Maßnahme einer Bedrohung zuordnen (Auszug) … Seite 59

Index

AIDE – 41, 42, 43, 61
AppArmor – 8, 25, 26, 27, 61
auditd – 38, 39, 40, 41
Backup – 8, 47, 48, 49, 50
Baseline – 9, 10, 11, 61
Capability – 51, 52, 53
cgroup – 51, 53
ClamAV – 44, 45, 46
CSP – 54, 56, 57
Docker – 51, 52, 53
Fail2ban – 8, 28, 29, 30, 31, 61
HSTS – 54, 56
Jail – 28, 29, 30, 31, 61
Namespace – 51, 53
nftables – 8, 18, 20, 22, 23, 28
Risikomatrix – 58, 59, 60
SUID – 12, 13, 14, 32, 61
sysctl – 35, 36, 37
TLS – 54, 55, 57
ufw – 22, 23, 24

LINUX SECURITY BAND 2: HARDENING

Vom Grundserver zum professionell gehärteten System

Band 2 einer dreibändigen Fachbuchreihe über Linux-Sicherheit unter Debian 13, direkte Fortsetzung von Band 1. Sechzehn Kapitel führen von CIS-Hardening-Standards über AppArmor, Fail2ban, Auditierung und Integritätsüberwachung bis zu Container- und Webserver-Sicherheit. Jedes Kapitel folgt derselben Struktur: Einführung, technische Grundlagen, Fachbegriffe, Praxis, Sicherheitsbetrachtung, zwei vollständige Laborübungen, typische Fehler, Zusammenfassung, Kontrollfragen und Lernzielkontrolle.

DAS ERWARTET DICH

- CIS-Hardening-Standards, vertiefte nftables- und ufw-Konfiguration
- AppArmor (Mandatory Access Control) und Fail2ban gegen automatisierte Angriffe
- Auditierung mit auditd, Integritätsüberwachung mit AIDE, Malware-Erkennung mit ClamAV
- Backup-Konzepte, Docker- und Container-Sicherheit, Webserver-Härtung
- 32 vollständige Laborübungen auf Debian 13, mit Musterlösungen und Kontrollfragen

Für wen?

Leser:innen von Band 1, die den dort aufgebauten Grundserver professionell weiter härten möchten. Grundlage für Band 3 (Analyse und Ethical Hacking).

64 Seiten · Stand Oktober 2026 · Band 2 von 3 · Debian 13 (Trixie)

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.